GradeFlow- College-focused Grading & Marks Management System

Project Documentation

Submitted in Partial fulfilment of the

Requirements for the award of the Degree of

MASTER OF SCIENCE (INFORMATION TECHNOLOGY)

By

SHRUTI SUDAY HARAYAN

Seat No. **202**

Under the esteemed guidance of

Prof. Suryaprakash Upadhyay

Assistant Professor

DEPARTMENT OF INFORMATION TECHNOLOGY

RAMANAND ARYA D.A.V COLLEGE, AUTONOMOUS

(Affiliated to University of Mumbai)

MUMBAI, 400042

MAHARASHTRA

2025-2026

RAMANAND ARYA D.A.V COLLEGE, AUTONOMOUS

(Affiliated to University of Mumbai)

MUMBAI, MAHARASHTRA – 400042

DEPARTMENT OF INFORMATION TECHNOLOGY

CERTIFICATE

This is to certify that project entitled, "**GradeFlow**", is a bonafide work of **SHRUTI SUDAY HARAYAN** bearing Seat No. **202** Submitted in the partial fulfilment of the requirements for the award of Degree of **MASTER OF SCIENCE in INFORMATION TECHNOLOGY** from the University of Mumbai.

Internal Guide **Coordinator**

External Examiner

Date : **College Seal**

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to all those who helped me complete this research successfully and I came to know about so many new things.

First and foremost, I would like to thank to **Dr. Balachandra shetty, Principal of Ramanand Arya DAV college,** who gave me golden opportunity to do this wonderful project.

I would like to express my special thanks and gratitude to the project guide **Assistant Professor Suryaprakash Upadhyay** for guiding me to do the project work on time and giving support and guidance, which made me complete my project duly. I am extremely thankful to him for providing such nice support and guidance.

I am thankful for and fortunate enough to get constant encouragement, support, and guidance from the teachers of Information Technology who helped me in successfully completing my project work.

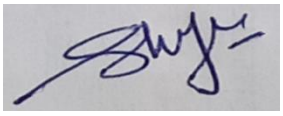
Finally, I would like to thank my family who motivated me and boosted my morale.

DECLARATION

I hereby declare that the project entitled, **"GradeFlow"** done at **RAMANAND ARYA DAV COLLEGE**, has not been in any case duplicated to submit to any other universities for the award of any degree. To the best of my knowledge, no one has submitted to any other university.

The project is done in partial fulfilment of the requirements for the award of degree of **MASTER OF SCIENCE (INFORMATION TECHNOLOGY)** to be submitted as final semester project as part of our curriculum.

SHRUTI S. HARAYAN



INDEX

Sr. No.	Chapter	Page no.	Sign
1	Introduction		
2	Description of Problem 2.1 research gaps		
3	Literature Review		
4	Research Methodology		
5	Code Implementation		
6	Conclusion		
7	Reference		

INTRODUCTION

1.1 Background

In many higher education institutions, the process of student assessment and mark consolidation is still predominantly managed using spreadsheet-based tools such as Microsoft Excel. While spreadsheets offer flexibility and ease of use, they are not purpose-built for academic evaluation workflows and often require extensive manual intervention. Teachers typically enter marks for individual questions or components and rely on manually defined formulas to compute totals and grand totals. This approach introduces a high risk of computational errors due to incorrect formulas, accidental cell modifications, or inconsistent application of evaluation rules.

Furthermore, spreadsheet-based grading lacks structured workflow control. In common practice, teachers submit completed Excel files to the examination cell, and any subsequently identified errors necessitate re-editing and re-submission of the same file. These revised files are often differentiated only through file naming conventions such as “final,” “updated,” or “revised,” which provides no formal mechanism to identify the authoritative version. As a result, examination committees frequently face difficulties in determining which submission represents the final and approved set of marks.

The absence of centralized control and automated rule enforcement in spreadsheet-based systems also affects consistency across evaluators. Complex evaluation schemes—such as selecting the best *N* scores from multiple sub-components—must be implemented manually, increasing variability and reducing reliability. From an administrative perspective, this leads to increased verification effort, delayed result processing, and a higher likelihood of discrepancies during moderation or result publication.

Recent research in academic information systems highlights the limitations of generic tools for assessment management and emphasizes the need for specialized, workflow-aware grading platforms that support automated computation, controlled submission states, and role-based access. However, many existing learning management systems prioritize content delivery and student interaction, offering limited flexibility for institution-specific grading rules and examination cell workflows. This gap necessitates the development of a dedicated grading system that addresses the operational realities of mark entry, validation, and finalization in higher education institutions.

1.2 Problem statement

Despite the widespread use of digital tools in higher education, the process of student mark entry and consolidation in many institutions continues to rely on spreadsheet-based solutions such as Microsoft Excel. These tools are not designed to support structured academic evaluation workflows and depend heavily on manual calculations and user-defined formulas. As a result, errors in total computation, inconsistent application of grading rules, and accidental data modifications are common.

Additionally, the absence of a controlled submission mechanism in spreadsheet-based systems leads to repeated re-editing and re-submission of mark sheets after initial submission. Revised files are often distinguished only by informal naming conventions, which creates ambiguity regarding the final and approved version of student marks. This lack of version control and submission finality causes confusion for examination cell committees and increases administrative overhead during result processing.

Existing learning management and student information systems provide limited support for fine-grained mark entry, complex aggregation rules, and institution-specific evaluation policies. Consequently, there is a need for a dedicated grading system that automates mark calculations, enforces consistent evaluation rules, and introduces a clear, workflow-driven submission process. Addressing these challenges is essential to improve accuracy, reduce administrative confusion, and ensure reliable management of academic assessment data.

1.3 Research Objectives

The primary objective of GradeFlow is to replace spreadsheet-based grading practices with a structured and reliable digital system for managing student assessment data. The specific objectives of the system are as follows:

- **To eliminate manual total calculations** by automating the computation of question-wise totals and grand totals based on predefined evaluation rules.
- **To enforce consistent grading rules** across all evaluators by supporting rule-based aggregation mechanisms, such as selecting the best *N* scores from multiple sub-components.
- **To provide fine-grained mark entry** at the sub-question level, enabling accurate and flexible representation of academic evaluation schemes.
- **To introduce a controlled submission workflow** that clearly distinguishes between draft and final mark submissions and prevents unauthorized or accidental re-editing after finalization.
- **To reduce administrative confusion** caused by multiple revised Excel files by maintaining a single authoritative version of student marks.
- **To support role-based access control**, ensuring that grading and submission actions are performed only by authorized users.
- **To improve the efficiency of examination processing** by minimizing verification effort and delays associated with manual error correction.

1.4 Motivation of the Study

The motivation for this study arises from the continued dependence on spreadsheet-based tools for managing student assessment in higher education institutions. Although spreadsheets provide flexibility and familiarity, they are not designed to support structured academic evaluation processes. The reliance on manual data entry and formula-based calculations often results in computational errors, inconsistent application of grading rules, and increased effort in verification and correction.

A significant challenge observed in existing practices is the lack of a controlled submission mechanism for student marks. Once marks are submitted in spreadsheet format, any detected error requires the file to be edited and resubmitted. The repeated creation of revised versions,

often distinguished only by informal file names, leads to ambiguity regarding the final and approved dataset. This ambiguity places additional burden on examination cell committees and increases the risk of inconsistencies during result compilation.

Furthermore, modern evaluation schemes frequently involve complex marking structures, such as multiple sub-questions and selective aggregation rules. Implementing these schemes accurately using spreadsheets demands careful manual configuration, which is both time-consuming and error-prone. These challenges highlight the limitations of generic tools in supporting institution-specific grading requirements.

The study is motivated by the need for a dedicated grading system that automates mark calculations, enforces consistent evaluation rules, and provides a clear workflow for mark submission and finalization. By addressing these issues, the proposed system aims to improve accuracy, reduce administrative confusion, and enhance the overall reliability of academic assessment management.

1.5 Scope of the Study

The scope of this study is limited to the design and implementation of a web-based grading system that supports structured mark entry, automated computation, and workflow-controlled submission for higher education institutions. The system focuses on examination-oriented assessment processes, including sub-question mark entry, rule-based aggregation, and role-based administrative control. The study does not address broader student information management functions such as admission processing, attendance tracking, or timetable management. Instead, it concentrates specifically on improving the reliability and clarity of academic grading workflows.

1.6 Structure of the Report

This report is organized into several chapters. Chapter 2 describes the problem in detail and identifies research gaps in existing grading approaches. Chapter 3 reviews related literature on academic assessment systems and grading tools. Chapter 4 explains the research methodology and system design approach. Chapter 5 presents the implementation of the GradeFlow system, while Chapter 6 discusses conclusions and future scope. References used in the study are listed in Chapter 7.

Description of Problem

In many higher education institutions, the process of student mark entry and consolidation is managed using spreadsheet-based tools such as Microsoft Excel. While these tools are widely available and easy to use, they lack built-in support for structured academic evaluation workflows. Teachers are required to manually enter marks for individual questions and configure formulas to calculate totals and grand totals. This manual approach increases the likelihood of calculation errors, inconsistencies, and accidental data modification.

Another critical issue is the absence of a formal mechanism to control the submission and revision of mark sheets. Once an Excel file is submitted to the examination cell, any subsequently identified mistake requires the same file to be edited and resubmitted. These revised submissions are commonly distinguished only by file names such as “final,” “updated,” or “corrected,” which do not provide reliable version identification. As a result, examination committees often face uncertainty in determining which file represents the final and approved set of marks.

Additionally, modern evaluation schemes frequently involve complex marking structures, including multiple sub-questions and selective aggregation rules such as choosing the best scores from a set of attempts. Implementing such rules manually in spreadsheets is time-consuming and error-prone, and it relies heavily on individual evaluator expertise. This lack of standardization can lead to inconsistencies across different evaluators and subjects.

From an administrative perspective, these challenges lead to increased verification effort, delays in result processing, and a higher risk of discrepancies during moderation and publication of results. The reliance on spreadsheets for grading therefore presents significant limitations in terms of accuracy, consistency, and workflow clarity. Addressing these issues requires a dedicated grading system that automates calculations, enforces evaluation rules, and provides a controlled mechanism for mark submission and finalization.

Beyond calculation errors and submission confusion, spreadsheet-based grading systems also suffer from limited transparency and traceability. Once marks are modified in a spreadsheet, it is often difficult to determine who made the change, when it was made, and why it was necessary. This lack of auditability creates challenges during moderation, re-evaluation requests, and accreditation audits, where institutions must demonstrate the integrity of their evaluation process.

Another issue arises from distributed responsibility among multiple teachers handling different divisions or batches of the same subject. In spreadsheet-based workflows, each teacher typically maintains a separate file, making it difficult for the examination cell to obtain a unified subject-level view. Consolidating these files requires manual merging, which increases the chances of duplication, omission, or inconsistent formatting.

The reliance on local spreadsheet files also introduces data security and accessibility concerns. Files may be stored on personal devices, shared via email, or transferred through messaging platforms, increasing the risk of accidental loss, unauthorized access, or outdated versions being circulated. In cases of system crashes or file corruption, recovery mechanisms are limited.

Furthermore, spreadsheet-based systems do not provide real-time workflow visibility. Administrators cannot easily track which teachers have completed mark entry, which subjects are pending, or which submissions are awaiting verification. This absence of centralized monitoring often leads to delays, repeated follow-ups, and last-minute data collection before result compilation.

2.1 Research Gap

Despite the increasing adoption of digital tools in education, the specific domain of academic assessment and mark management remains inadequately addressed by both general-purpose and specialized systems. A review of the literature and current practices reveals several key gaps in existing solutions:

1. Limited Support for Structured Evaluation Workflows

Current spreadsheet-based approaches, while flexible, do not enforce structured workflows for entering, validating, and submitting marks. This lack of workflow control results in ambiguous document states, where multiple versions of mark sheets exist without clear distinction between draft and final submissions. Examination cells often receive numerous files with inconsistent naming conventions, leading to confusion and inefficiencies in result processing.

2. Manual Computation and Error Susceptibility

Spreadsheets require teachers to manually define and maintain formulas for computing totals and grand totals. Such reliance on manual calculations introduces a high risk of errors due to incorrect formulas, accidental overwriting, or misconfiguration of cell logic. These computation errors can propagate through result publication, undermining data accuracy and requiring additional verification effort.

3. Inadequate Support for Complex Grading Rules

Many modern evaluation schemes involve complex aggregation logic—such as selecting the best N marks from a set of sub-components, applying weights, or enforcing conditional constraints. Existing systems, especially spreadsheets and basic gradebooks in learning management systems (LMS), do not natively support these rules. Implementing such logic manually is time-consuming, non-scalable, and prone to inconsistency across evaluators.

4. Lack of Consistency and Standardization

Spreadsheet workflows are highly individualized; teachers may use different structures, formulas, and naming conventions. This variability leads to non-standardized data formats, making it difficult for examination cells to automate downstream processes such as result verification, moderation, and publication.

5. Absence of Controlled Submission and Versioning

Existing systems lack mechanisms to distinguish between draft, reviewed, and final submissions in a verifiable way. While some LMS platforms include gradebooks, they do not

typically offer explicit version control or lock-state transitions for mark sheets. As a result, revised mark sheets can circulate without authoritative status, increasing administrative overhead and reducing trust in the evaluation process.

6. Disconnect Between Evaluation Practices and Digital Tool Capabilities

The gap between institutional grading needs and tool capabilities is evident in how educators persist with spreadsheets despite the availability of digital tools. This indicates that current systems either do not fully align with institutional workflows or lack functionality that practical grading scenarios require.

7. Lack of Audit Trails and Change Tracking

Existing grading practices do not maintain detailed logs of modifications made to marks. Without an audit trail, institutions cannot easily verify the authenticity of updates or resolve disputes related to mark changes.

8. Difficulty in Multi-Evaluator Consolidation

When multiple teachers handle different divisions or components of the same subject, there is no built-in mechanism to automatically combine their marks into a unified dataset. This results in manual consolidation work for administrators.

9. Weak Data Security and Centralized Control

Spreadsheet files are often exchanged through informal channels such as email or messaging platforms. This approach lacks centralized storage, controlled access permissions, and institutional-level data protection.

10. Limited Administrative Monitoring Tools

Existing approaches do not provide dashboards or indicators to show submission progress, pending evaluations, or locked mark sheets. This reduces the examination cell's ability to manage timelines effectively.

Why This Gap Matters for GradeFlow

These gaps collectively motivate the need for a dedicated grading system that:

- **Automates mark calculations** using configurable aggregation rules
- **Enforces structured workflows** with clear draft and final submission states
- **Supports complex evaluation logic** natively
- **Ensures consistency and standardization** across evaluators
- **Reduces administrative workload** and confusion for examination cells

GradeFlow is designed to fill these gaps by combining rule-based computation, controlled submissions, and a unified interface tailored to academic assessment workflows.

Literature Review

3.1 Introduction to the Literature Review

The management of academic assessment data is a critical component of higher education administration, as it directly impacts the accuracy, reliability, and timeliness of student results. Over the years, various digital tools and systems have been adopted to support grading and evaluation processes. However, despite advancements in educational technology, many institutions continue to rely on spreadsheet-based methods and partially integrated systems for managing student marks.

Existing literature highlights that while spreadsheets and generic digital tools offer flexibility, they are not designed to support structured assessment workflows or enforce consistent evaluation rules. Studies on academic information systems emphasize that manual computation and informal version control mechanisms contribute significantly to grading errors and administrative inefficiencies. Furthermore, research on learning management systems indicates that their gradebook components often prioritize learning analytics and continuous assessment rather than examination-oriented mark processing.

The literature review aims to examine existing approaches to student assessment management, including spreadsheet-based grading practices, learning management system gradebooks, and specialized academic information systems. By analyzing their strengths and limitations, this review identifies gaps related to automated computation, workflow control, and support for complex grading schemes. Understanding these gaps is essential for motivating the development of a dedicated grading system such as GradeFlow.

This review therefore provides the theoretical and practical foundation for the proposed system by situating it within existing research and identifying areas where current solutions fall short. The insights derived from the literature guide the design decisions and validate the need for a rule-driven, workflow-controlled grading platform tailored to higher education institutions.

The reviewed literature can be broadly grouped into three categories: studies examining spreadsheet reliability, research on learning management system gradebooks, and work on workflow-oriented academic information systems. The following sections synthesize findings from these areas to establish the limitations of existing solutions and identify requirements for improved grading platforms.

3.2 Literature Review

1. Spreadsheet-Based Systems for Academic Grading

Spreadsheets such as Microsoft Excel have long been used for managing student marks due to their accessibility, flexibility, and low learning curve. Several studies note that spreadsheets are commonly adopted in academic environments for data entry, numerical computation, and result preparation (Panko, 2008). However, research consistently highlights

that spreadsheets are not purpose-built for assessment management and are highly susceptible to human error.

Panko (2008) reports that a significant percentage of operational spreadsheets contain non-trivial errors, often arising from incorrect formulas, misreferenced cells, or accidental overwriting of data. In the context of academic grading, such errors can directly affect student results and require extensive verification efforts. Additionally, spreadsheet-based systems lack built-in mechanisms for enforcing evaluation rules, resulting in inconsistent implementation of grading logic across evaluators.

Another limitation identified in the literature is the absence of structured version control in spreadsheet workflows. As noted by Rajalingham et al. (2005), spreadsheet users often rely on informal file naming conventions to manage revisions, which leads to ambiguity regarding the most recent or authoritative version. This issue is particularly problematic in examination workflows, where clarity and finality of submitted marks are essential.

2. Manual Computation and Error Propagation in Grading Processes

Manual computation remains a central concern in spreadsheet-based grading systems. While spreadsheets can automate calculations through formulas, the responsibility for creating and maintaining these formulas lies with the user. Research by Powell et al. (2009) demonstrates that even experienced users frequently introduce logical or structural errors when building complex computational models in spreadsheets.

In academic assessment scenarios involving multiple sub-questions and selective aggregation rules, the complexity of formulas increases significantly. Errors introduced at this stage can propagate through totals and grand totals, often remaining undetected until late stages of result processing. This increases administrative workload and delays result publication, as highlighted in studies on academic process inefficiencies (Ellis, 2013).

3. Learning Management Systems and Gradebook Limitations

Learning Management Systems (LMS) such as Moodle, Canvas, and Google Classroom have become integral to modern education. These systems provide digital gradebooks that automate basic calculations and centralize assessment data. Research acknowledges that LMS gradebooks reduce some of the risks associated with manual calculations (Dawson et al., 2014).

However, multiple studies point out that LMS gradebooks are primarily designed for continuous assessment and learning analytics rather than examination-oriented workflows. Dawson et al. (2014) note that LMS gradebooks often lack flexibility in representing institution-specific grading schemes and provide limited support for complex aggregation rules such as selecting the best *N* scores from multiple components.

Furthermore, LMS platforms typically allow grades to be edited even after publication, with limited enforcement of final submission states. This weak notion of finality is unsuitable for

high-stakes examination processes, where post-submission modifications must be tightly controlled. As a result, institutions often continue to rely on external tools or manual processes alongside LMS platforms.

4. Workflow Control and Submission Finality in Assessment Systems

Workflow management is a critical aspect of reliable assessment systems. Literature on academic information systems emphasizes the importance of clearly defined states for data entry, review, approval, and finalization (Laudon & Laudon, 2016). In the absence of such workflows, systems fail to provide accountability and clarity.

Existing spreadsheet-based and LMS-based solutions lack explicit mechanisms to distinguish between draft and final submissions of marks. As observed by Behl and Behl (2017), the absence of enforced submission finality leads to repeated revisions, increased communication overhead, and confusion among administrative stakeholders. This issue is particularly evident in examination cell operations, where multiple revised mark sheets may circulate simultaneously.

5. Need for Rule-Driven and Examination-Oriented Grading Systems

Several researchers argue for the development of specialized academic systems that go beyond generic data management tools. Ellis (2013) emphasizes that assessment systems must align with institutional policies and evaluation practices to be effective. Generic tools, while flexible, often fail to encode complex grading rules in a consistent and reusable manner.

Rule-driven systems, where evaluation logic is explicitly defined and automatically applied, have been shown to reduce variability and improve reliability in assessment outcomes (Shute & Kim, 2014). Such systems ensure that all evaluators apply grading policies uniformly, minimizing subjective or procedural inconsistencies.

Despite this recognition, there is limited adoption of dedicated grading platforms that focus specifically on examination workflows, controlled submissions, and automated rule enforcement. This gap in existing literature and practice motivates the development of systems like GradeFlow.

6. Summary of Literature and Identified Gap

The literature indicates that while spreadsheets and LMS gradebooks are widely used for managing student assessment data, both suffer from significant limitations. Spreadsheet-based systems are prone to manual computation errors and lack structured workflow control, while LMS gradebooks provide limited flexibility for complex grading schemes and insufficient enforcement of submission finality.

There is a clear research gap in the availability of dedicated grading systems that combine automated, rule-based computation with explicit workflow management tailored to

examination processes. Addressing this gap is essential to improve accuracy, consistency, and administrative efficiency in academic assessment management. GradeFlow is proposed as a response to this gap, drawing on insights from existing research and addressing the practical limitations identified in the literature.

Comparative Analysis of Existing Systems and GradeFlow

Academic assessment and mark management systems play a critical role in ensuring accuracy, consistency, and transparency in result processing. However, existing tools—primarily spreadsheet-based systems and generic Learning Management System (LMS) gradebooks—exhibit several limitations when applied to real-world examination workflows. This section presents a comparative analysis of existing systems and the proposed GradeFlow system, highlighting functional gaps and illustrating how GradeFlow addresses them.

1. Spreadsheet-Based Grading Systems (Excel)

Spreadsheet tools such as Microsoft Excel are widely used due to their availability and flexibility. Teachers typically enter marks manually and define formulas for calculating totals and grand totals. While spreadsheets support tabular data entry, they lack built-in mechanisms for enforcing structured academic workflows.

Limitations

- **Manual computation dependency:** Total and aggregate marks rely on manually created formulas, increasing the likelihood of errors due to incorrect references or accidental overwrites.
- **Lack of workflow states:** There is no distinction between draft and final submissions. Any modification after submission results in another edited file.
- **Version ambiguity:** Revised mark sheets are commonly distinguished by informal file naming conventions (e.g., *final.xlsx*, *final_updated.xlsx*), making it difficult for examination cells to identify the authoritative version.
- **Inconsistent rule implementation:** Complex grading rules (e.g., selecting best *N* scores) are implemented differently by different evaluators, leading to inconsistencies.

Example Scenario

Consider a question with five sub-parts where only the best three marks must be counted. In Excel, the teacher manually creates formulas to sort and sum values. If a cell reference is incorrect or later edited, the total becomes invalid, and the error may remain unnoticed until result verification. Similar risks have been highlighted in studies examining spreadsheet-based academic data processing [1].

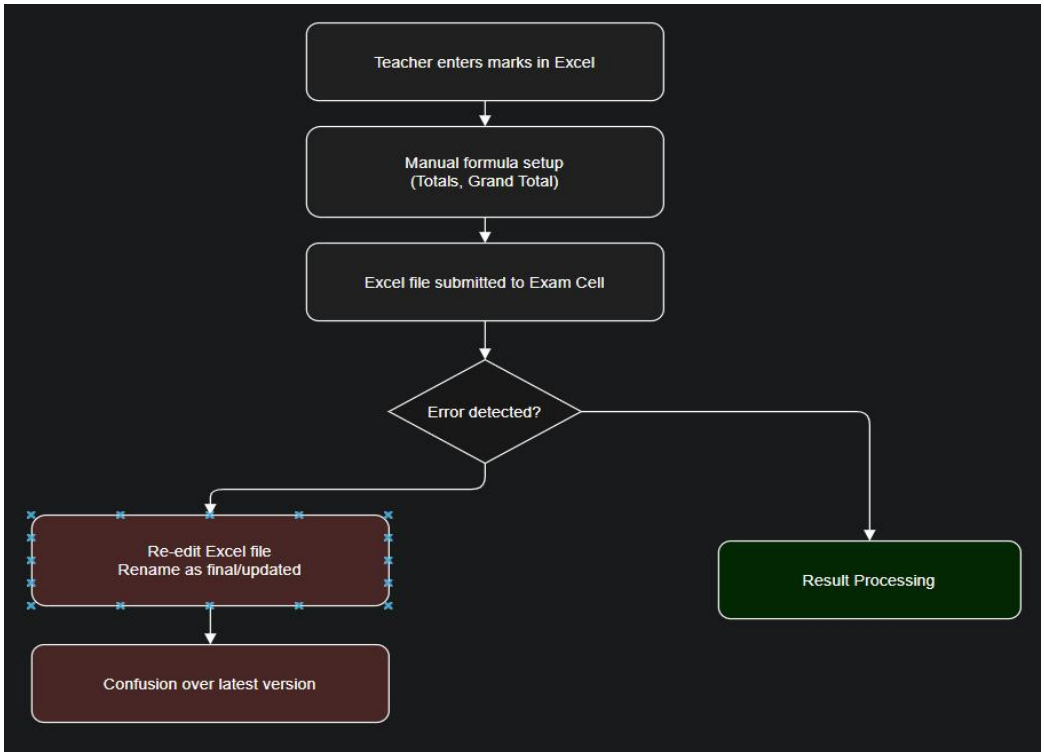


Figure 1: Existing spreadsheet-based grading workflow showing manual calculations, repeated file revisions, and ambiguity caused by multiple versions of submitted mark sheets.

2. LMS Gradebooks (Moodle, Google Classroom, Canvas)

Modern LMS platforms include gradebook functionalities and provide better structure than spreadsheets. However, their primary focus is on learning delivery rather than examination-oriented grading workflows.

Strengths

- Automated total calculation for basic grading schemes
- Centralized data storage
- Role-based access at a coarse level

Limitations

- **Limited flexibility for complex evaluation rules:** Many LMS gradebooks support weights and averages but struggle with rules such as "best *N* out of *M*" without plugins or workarounds.
- **Insufficient workflow enforcement:** Grades can often be edited even after being published, and the notion of a locked, final submission is weak or absent.
- **Misalignment with exam cell workflows:** LMS gradebooks are designed for continuous assessment rather than high-stakes examination processing.

Example Scenario

If an evaluator mistakenly enters an incorrect mark and later edits it after grades are released, the LMS may allow the change without clearly signaling finality or approval status to the examination committee. Prior research notes that LMS gradebooks often lack explicit lifecycle states for assessment data [2].

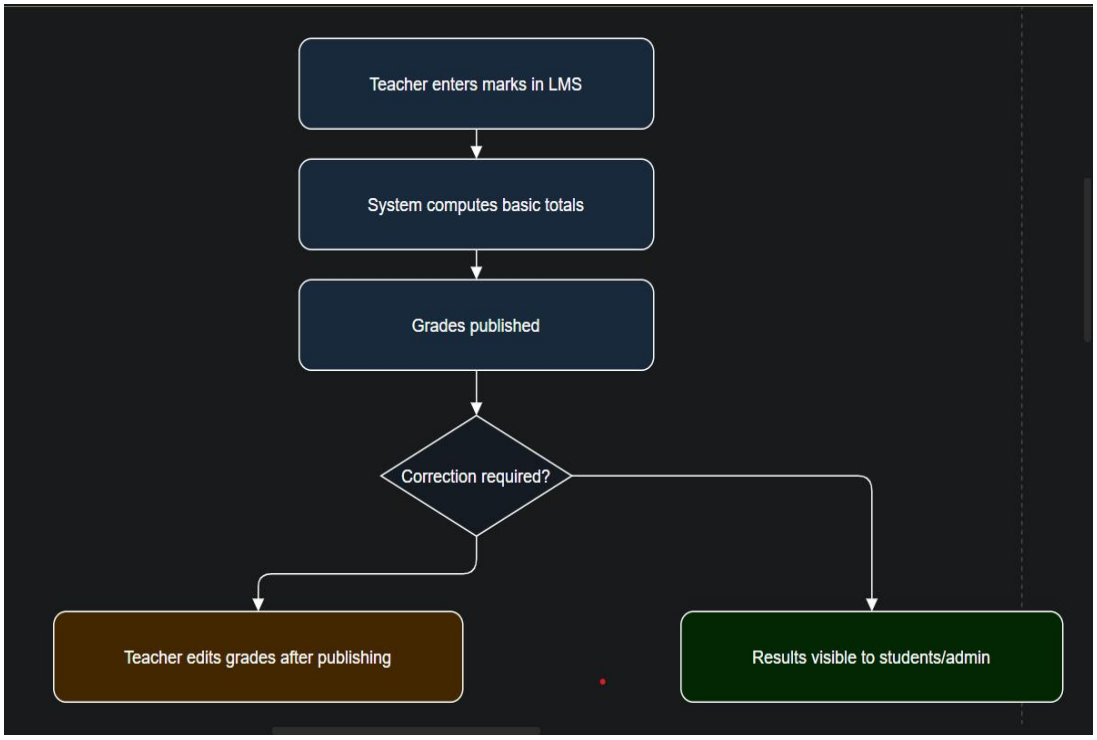


Figure 2: LMS gradebook workflow demonstrating automated calculation but limited enforcement of final submission or lock states for examination data.

3. GradeFlow: Proposed System

GradeFlow is designed specifically to address the shortcomings of both spreadsheets and generic LMS gradebooks by focusing on **rule-driven computation and workflow control**.

Key Capabilities

- **Automated, rule-based calculations:** All totals and aggregates are computed server-side using predefined rules, eliminating manual formula errors.
- **Support for complex grading schemes:** Evaluation logic such as selecting the best N sub-question marks is implemented as a first-class system feature.
- **Controlled submission workflow:** GradeFlow explicitly distinguishes between draft and final submissions. Once finalized, marks are locked and cannot be edited.
- **Single source of truth:** The system maintains one authoritative dataset, eliminating confusion caused by multiple revised files.
- **Role-based enforcement:** Teachers and administrators have clearly defined permissions aligned with institutional processes.

Example Scenario

Using the same "best 3 out of 5" question, GradeFlow allows the evaluator to enter all five sub-question marks. The system automatically selects the top three values and computes the total consistently for all students. Once the evaluator finalizes the submission, the system locks the data, ensuring that the examination cell accesses only the approved version.

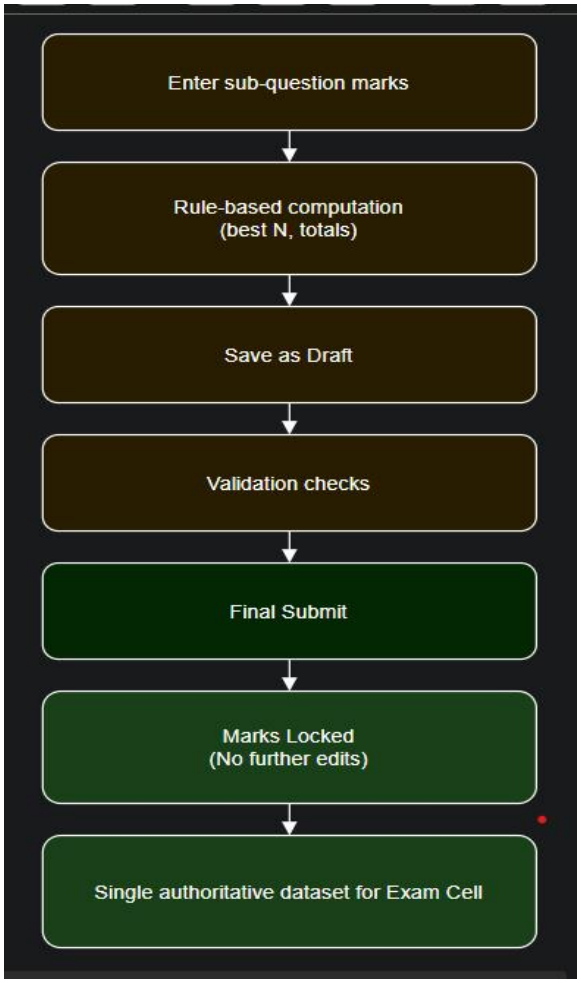


Figure 3: Proposed GradeFlow workflow illustrating rule-based computation, controlled submission states, and enforced finalization of marks.

4. Comparative Summary

Aspect	Spreadsheets	LMS Gradebooks	GradeFlow
Calculation reliability	Low (manual formulas)	Medium	High (automated rules)
Workflow control	None	Limited	Explicit & enforced
Support for complex rules	Ad-hoc	Partial	Native
Version clarity	Poor	Moderate	Clear single version
Exam-oriented design	No	Partial	Yes

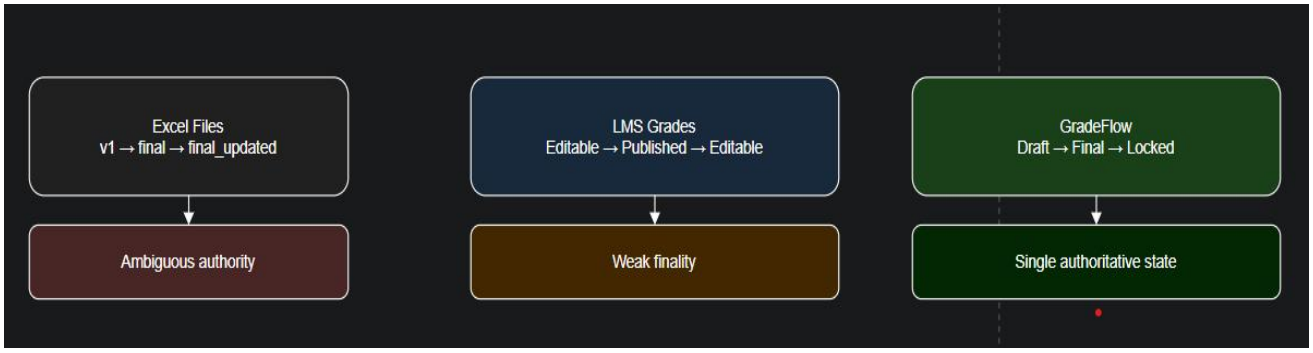


Figure 4: Comparison of version control mechanisms across spreadsheet systems, LMS gradebooks, and the proposed GradeFlow system.

5. Identified Research Gap

Existing systems either prioritize flexibility (spreadsheets) or learning delivery (LMS platforms), but neither adequately supports structured, rule-driven examination workflows. The literature emphasizes the need for specialized assessment management tools that reduce human error, enforce consistency, and support institutional evaluation practices [1][3]. GradeFlow addresses this gap by integrating automated computation with controlled submission states tailored to academic grading requirements.

Comparison: Existing Systems vs GradeFlow

Feature / Capability	Typical Spreadsheets (Excel)	Common LMS Gradebooks (Moodle / Canvas / Google)	GradeFlow (proposed)
Workflow control (draft → final → locked)	✖ No built-in submission states; manual file handover.	✗ Limited; gradebooks allow edits and "publish" but lack enforced lock/version semantics.	✅ Explicit draft & final states; finalization locks marks to prevent re-editing.
Rule-based aggregation (e.g., best *N* of *M*)	✖ Implemented via custom formulas; error-prone and non-standard.	✗ Basic weighting supported; complex rules need workarounds or plugins.	✅ First-class, declarative grading rules (take-best N, weights, conditional rules).
Sub-question granularity	✅ Possible but unstandardized (custom spreadsheets).	✅ Supported to varying degrees; structures differ by LMS.	✅ Native support for sub-question marking and mapping to main questions.
Automated, consistent calculations	✖ Dependent on user formulas; susceptible to mistakes/overwrites.	✅ Automated but sometimes limited for custom aggregation logic.	✅ Server-side computation; deterministic and reproducible results.
Versioning & authoritative single source	✖ Multiple files with ambiguous names (final/updated) — no reliable single source.	✗ Some history exists but not focused on multiple file submissions workflow.	✅ Single authoritative dataset with clear version/status; eliminates ambiguous file re-submissions.

Feature / Capability	Typical Spreadsheets (Excel)	Common LMS Gradebooks (Moodle / Canvas / Google)	GradeFlow (proposed)
Validation & error prevention (live checks)	✖ Minimal (cell validation only); no centralized validation.	✓ Some validations available; limited custom checks.	✅ Live validation rules (missing marks, range checks, required sections) before finalization.
Role-based access (teacher/admin)	✖ Managed externally (file sharing/permissions) — not integrated with grading logic.	✅ Role support exists but often coarse-grained.	✅ Fine-grained role enforcement integrated with workflow (teacher vs admin behaviors).
Bulk operations (import/export) & templates	✅ CSV/Excel friendly but inconsistent templates across teachers.	✅ Supports imports; templates may be rigid.	✅ Standardized import/export with template validation to reduce format variance.
Reproducibility & audit friendliness (recompute totals reliably)	✖ Hard to reproduce if formulas changed or files edited.	✓ More reproducible but complex rules may still vary.	✅ Reproducible server-side rule evaluation; same inputs → same outputs.
Ease of adoption / training	✅ Familiar to teachers but habits lead to errors.	✓ Familiar if institution uses LMS; may not match exam workflows.	✅ Intuitive UI modeled on existing workflows with reduced room for user errors (some training required).
Customization for institutional rules	✓ Possible but ad-hoc and distributed.	✗ Varies; often requires plugins or admin config.	✅ Designed to encode and reuse institution-specific evaluation policies as declarative objects.

Concluding Remarks on Literature

The literature consistently indicates that existing grading tools either provide flexibility without control (spreadsheets) or structure without workflow specificity (LMS gradebooks). None fully address the combined requirements of automated rule enforcement, submission finality, institutional workflow alignment, and centralized administrative control. These findings justify the design direction adopted in GradeFlow and establish the need for a dedicated, examination-oriented grading platform.

Research Methodology

1. Research Approach

This study adopts a **design-oriented and applied research approach**, focusing on the identification of practical problems in academic grading workflows and the development of a software solution to address those problems. The research emphasizes understanding existing grading practices, analyzing their limitations, and proposing a system that improves accuracy and workflow clarity.

A **problem-driven methodology** is followed, where real-world challenges observed in spreadsheet-based grading systems form the basis for system design and implementation.

This research follows a **Design Science Research (DSR) perspective**, where the primary objective is to create and evaluate an artifact that solves a real institutional problem. The artifact in this study is the GradeFlow system. The methodology therefore combines problem investigation, artifact design, implementation, and evaluation. Instead of purely theoretical modeling, the study emphasizes a practical solution whose effectiveness can be observed through workflow improvements and error reduction in grading processes.

The study is qualitative in nature with a system-development component, as it relies on institutional workflow understanding rather than large-scale statistical datasets. The focus is on improving operational processes rather than predicting user behavior.

Hypothesis

This study is based on the assumption that replacing spreadsheet-based grading workflows with a rule-driven and workflow-controlled grading system can improve the accuracy, consistency, and administrative efficiency of academic assessment management.

H_1: Automated rule-based computation in GradeFlow reduces calculation errors in totals and grand totals compared to spreadsheet-based grading.

H_2: Structured workflow states (draft, final, and locked) reduce ambiguity caused by multiple revised mark submissions.

H_3: Built-in support for complex grading rules (e.g., best-N selection, sub-question aggregation) improves consistency across evaluators and subjects.

H_4: Role-based access control and centralized storage increase accountability and prevent unauthorized post-submission modifications.

H_5: Use of GradeFlow reduces administrative verification effort and result processing time in examination workflows.

2. Identification of the Problem

The initial phase of the study involved analyzing the current grading practices used in higher education institutions. Informal observations and interactions with teaching staff revealed that:

- Marks were entered using Excel spreadsheets
- Total calculations were performed manually or via user-defined formulas
- Revised mark sheets were frequently re-submitted due to errors
- File renaming conventions caused confusion for examination committees

These observations helped define the core problem and establish the functional requirements of the proposed system.

Data Sources for Problem Identification

The problem definition was informed by:

- Discussions with teaching staff regarding grading practices
- Examination cell procedures for collecting and verifying marks
- Observation of spreadsheet templates used for internal assessment
- Review of institutional grading formats and submission patterns

This multi-source understanding ensured that the system requirements were grounded in real academic workflows rather than hypothetical assumptions.

3. Literature Review and Gap Analysis

A comprehensive review of existing literature was conducted to understand current approaches to academic grading and assessment management. This included studies on:

- Spreadsheet error rates and limitations
- LMS gradebook functionalities
- Workflow management in academic information systems

The review revealed a gap between existing tools and the requirements of examination-oriented grading workflows, particularly in terms of automated rule enforcement and submission finality. This gap directly informed the design objectives of GradeFlow.

4. System Design Methodology

The proposed system, GradeFlow, was designed using a **modular and iterative development approach**. The design phase focused on:

- Defining system roles (teachers and administrators)
- Designing workflows for mark entry, validation, and final submission
- Modeling grading rules for automated computation
- Ensuring clear separation between draft and final states

System diagrams and workflow models were created to visualize existing processes and the proposed solution. The design phase also incorporated **user-centered considerations**, ensuring that the system interface closely resembles familiar spreadsheet workflows while

introducing structured controls. This approach reduces resistance to adoption and minimizes the learning curve for teachers.

To ensure maintainability, the system architecture was separated into presentation, logic, and data layers. This layered design allows future modifications to grading rules or workflows without requiring major structural changes to the system.

5. Development and Implementation

GradeFlow was implemented as a **web-based application** using modern full-stack development practices.

Implementation steps included:

- Designing database structures to store marks at sub-question level
- Implementing rule-based computation logic for totals and grand totals
- Developing role-based access control mechanisms
- Creating user interfaces for mark entry, saving drafts, and final submission
- Enforcing system-level locks to prevent post-finalization edits

Version-wise development was followed, where core functionality such as mark storage and calculation was implemented first, followed by workflow enforcement features such as draft saving, final submission, and admin locking. This phased implementation ensured that foundational accuracy was verified before introducing workflow constraints.

Security considerations were incorporated during development, including validation checks, restricted edit permissions after submission, and controlled administrative overrides. These measures ensure that the system not only computes marks correctly but also protects their integrity.

6. Validation and Testing

The system was validated using **scenario-based testing**, reflecting real-world grading situations. Test cases included:

- Entry of marks for multiple sub-questions
- Application of different grading rules (e.g., best N out of M)
- Detection of missing or invalid marks
- Attempted edits after final submission

Results were compared with manually calculated outcomes to verify correctness and consistency. In addition to functional testing, usability checks were performed to ensure that the system could be operated intuitively by teachers familiar with spreadsheet-based workflows. Feedback from trial usage helped refine input validation messages, workflow prompts, and screen navigation.

Edge-case scenarios were included in testing, such as incomplete mark entry, duplicate roll numbers, and late corrections after final submission. These tests ensured that the system behaves predictably under non-ideal conditions.

7. Evaluation Criteria

The effectiveness of GradeFlow was evaluated based on the following criteria:

- **Accuracy:** Correct computation of totals and grand totals
- **Consistency:** Uniform application of grading rules
- **Usability:** Ease of use for teachers transitioning from Excel
- **Workflow clarity:** Clear distinction between draft and final submissions
- **Error reduction:** Elimination of manual calculation and versioning errors
- T**ransparency:** Ability of the system to clearly show submission state and mark status
- **Traceability:** Capability to track when marks were finalized and locked
- **Administrative control:** Ease with which examination staff can monitor submissions and enforce rules.
- **Scalability potential:** Suitability of the system structure for future expansion to additional departments or institutions

8. Tools and Technologies Used

- **Frontend – TypeScript, React, Tailwind CSS**
  The user interface is built using React with TypeScript for better code reliability and maintainability. Tailwind CSS is used to design a responsive, clean, and consistent UI across the system.
- **Backend – FastAPI (Python)**
  FastAPI is used to develop the backend APIs, enabling fast performance, automatic request validation, and efficient handling of exam, marks, and user management operations.
- **Database – SQLite (Relational Database)**
  SQLite is used to store structured academic data such as programmes, courses, students, exams, and marks, ensuring reliable storage and easy querying.
- **Diagramming – draw.io (diagrams.net)**
  Used to create system architecture diagrams, workflows, and database design visuals for documentation and better system understanding.
- **Testing – Manual and Scenario-Based Testing**
  The system was tested using real academic scenarios to verify workflows like marks entry, draft saving, final submission, locking, exporting reports, and role-based access control.

9. Methodological Limitations

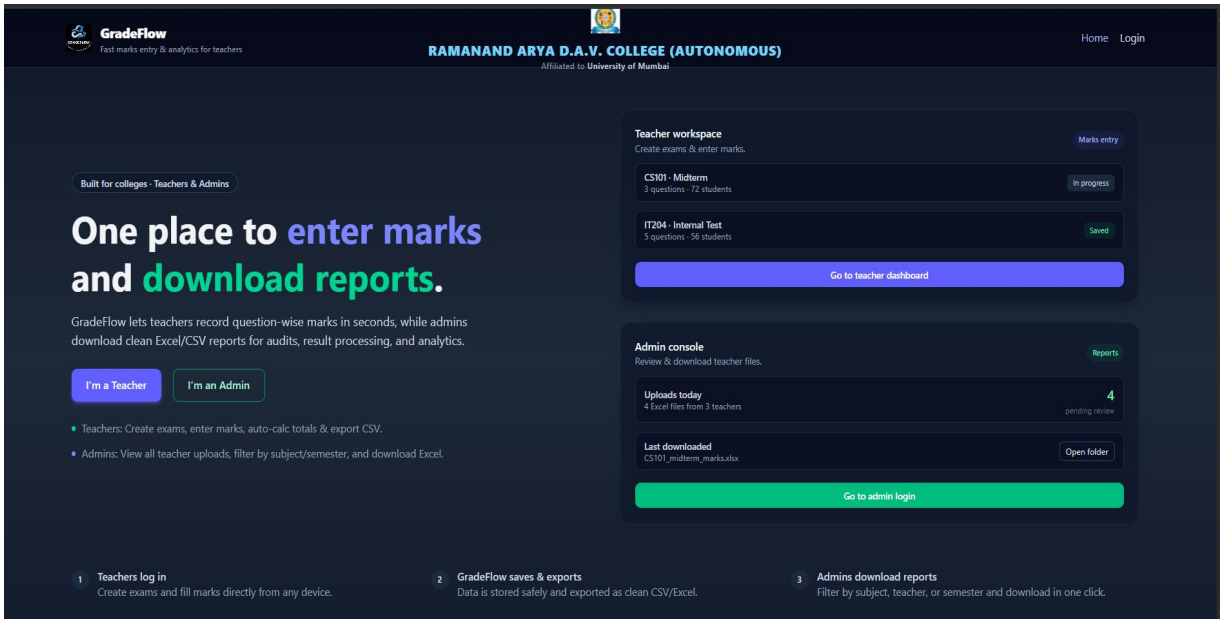
This study focuses on system design and workflow improvement within a limited institutional context. Large-scale deployment across multiple universities was not part of the evaluation scope. As a result, usability findings are indicative rather than statistically generalized. Future research may involve broader institutional trials and quantitative measurement of efficiency gains.

10. Methodology Summary

Overall, the research methodology integrates problem analysis, system design, implementation, and evaluation into a unified framework. By grounding the development of GradeFlow in real institutional practices and validating it through scenario-based testing, the study ensures that the proposed solution is both practically relevant and methodologically sound.

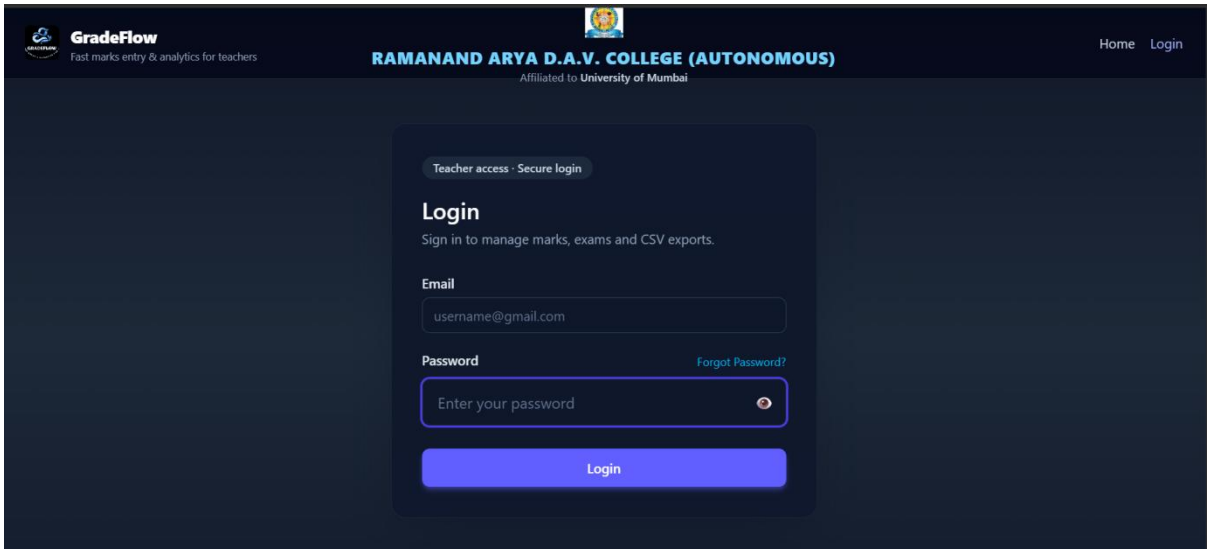
Code Implementation

1. GradeFlow Home Page



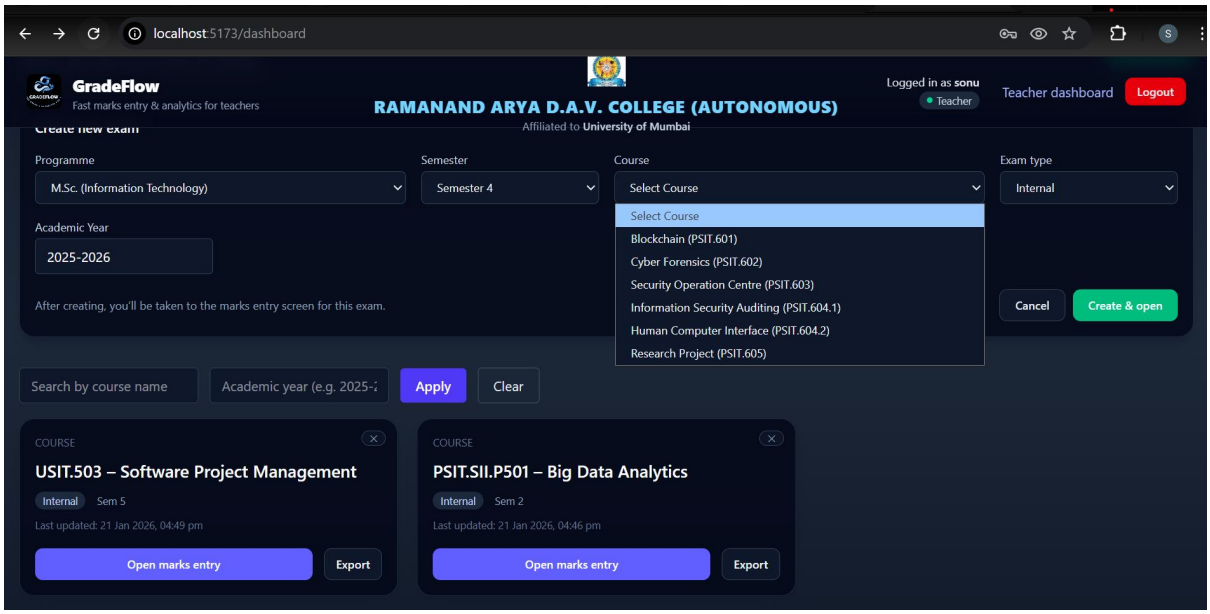
This screenshot shows the GradeFlow home page designed for teachers and administrators. It highlights centralized marks entry, automatic total calculation, and report downloads. Separate access points for teacher and admin roles are provided to ensure clear workflow management.

2. GradeFlow Login Page



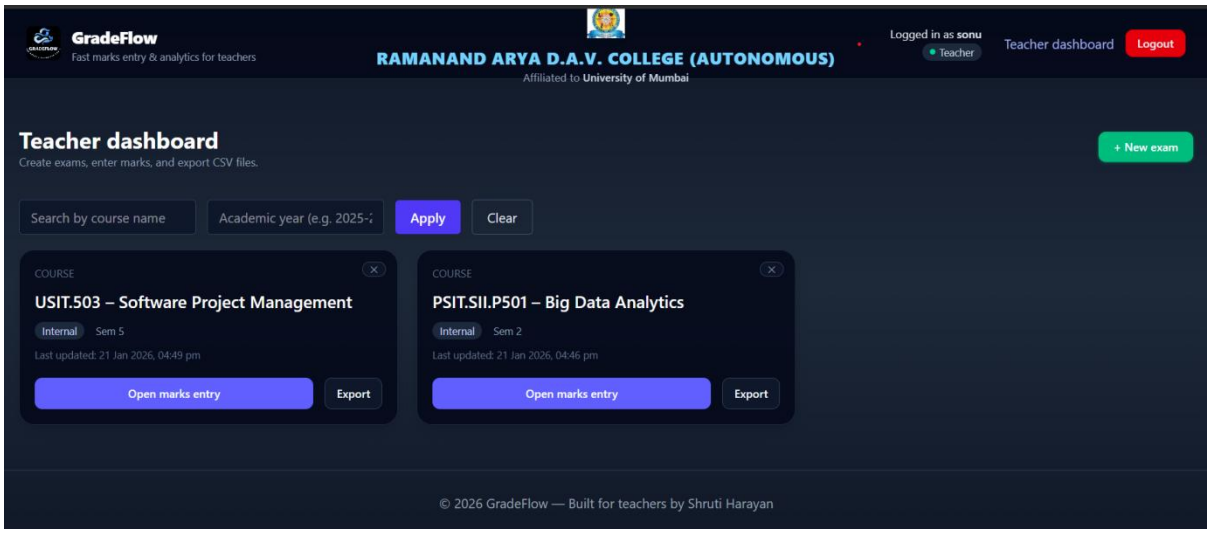
This screenshot displays the **secure login interface** of GradeFlow. It allows authorized teachers to access exam creation, marks entry, and data export features. The interface emphasizes simplicity, usability, and secure authentication.

3. Create New Exam Form



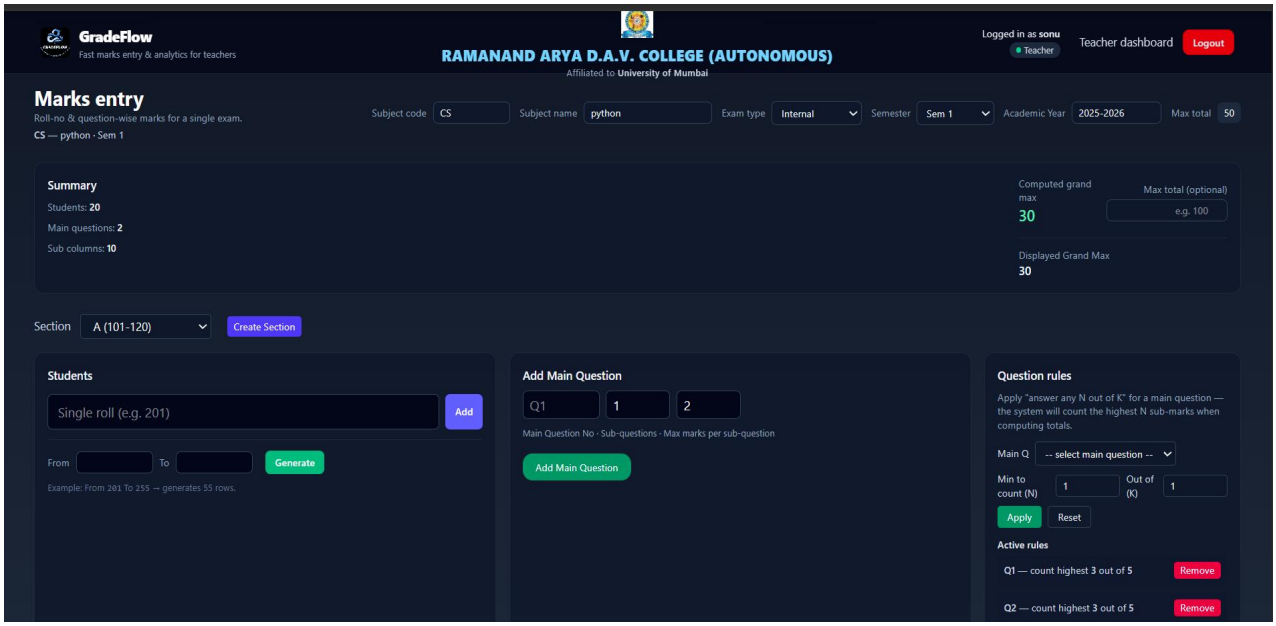
This screen is the **Teacher Dashboard** in GradeFlow, used to create and manage exams. Teachers select the programme, semester, course, exam type, and define academic year to create a new exam and directly open the marks entry screen. It also displays a list of existing exams with options to open marks entry or export exam data.

4. Teacher Dashboard – Exam Overview



Here, teachers can view all created subjects and exams. It provides filtering options by subject name and academic year. Each exam card allows quick access to marks entry and CSV export.

5. Marks Entry Setup Screen



This image displays the **marks entry configuration screen** for a selected exam. It shows subject details, exam type, semester, academic year, and computed grand total. Teachers can manage sections, students, main questions, and apply evaluation rules.

Student Entry Methods

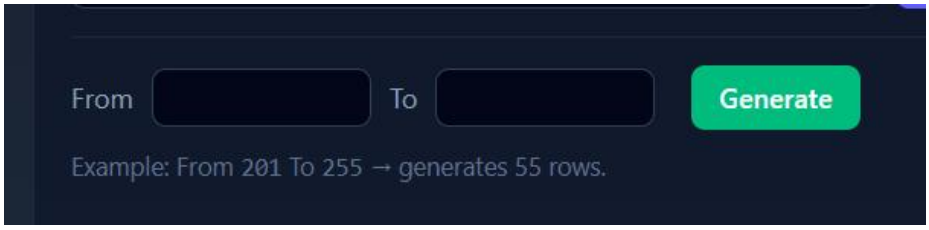
GradeFlow supports **three ways to add students**, allowing teachers to work according to their existing records.

METHOD 1 — Roll Number Range Generation

If the teacher has a full class list, they do not need to enter students individually.
They simply enter the starting and ending roll numbers and click **Generate**.

For example, if roll numbers are from **201 to 220**, the system automatically creates all rows instantly.

This saves time and prevents manual typing errors.



METHOD 2 — Section-Based Organization

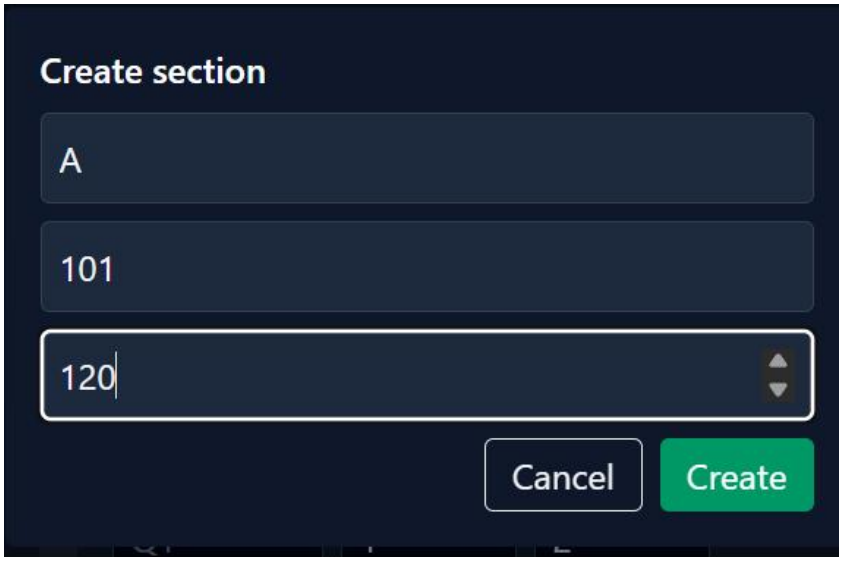
Here, the teacher selects the **section or division** of the class.
This allows marks to be managed separately for each class division, such as A, B, or C.

For example, Section A may contain roll numbers from **101 to 120**.
The system keeps sections separate so that attendance, roll numbers, and marks remain organized.

Since roll numbers belong to a specific section, the system stores them under that section.
This helps generate section-wise reports and prevents mixing students from different divisions.

NOTE: Sections are optional.
They are used only when a programme has multiple divisions. For example BSc. IT with Section A and B, then the teacher should create students under the correct section.

If there is only one class, teachers can directly generate roll numbers without creating sections.

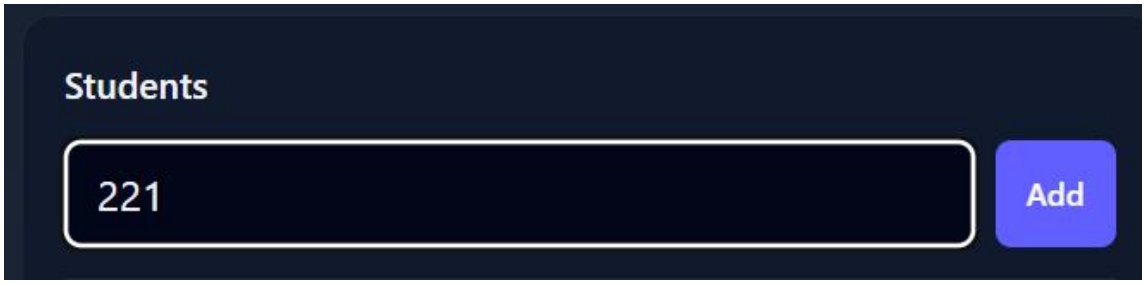


METHOD 3 — Single Roll Number Entry

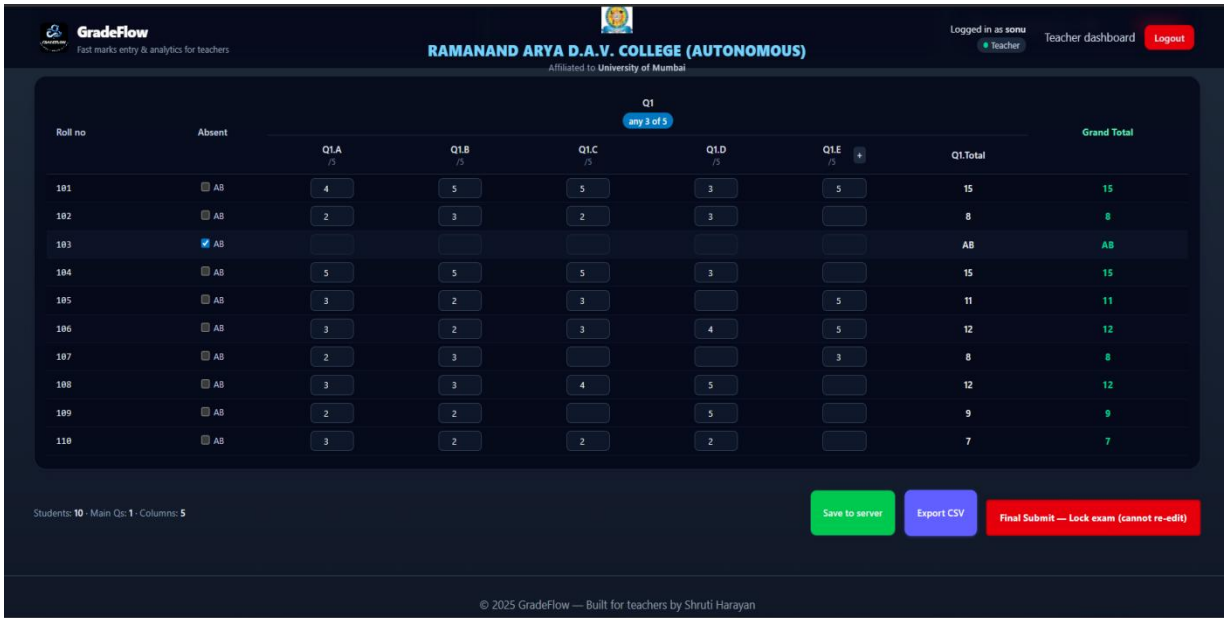
If a teacher wants to add one student manually, they can enter a single roll number and click **Add**.

This is useful when:

- A new student joins late
- One record needs correction
- A student was missed earlier

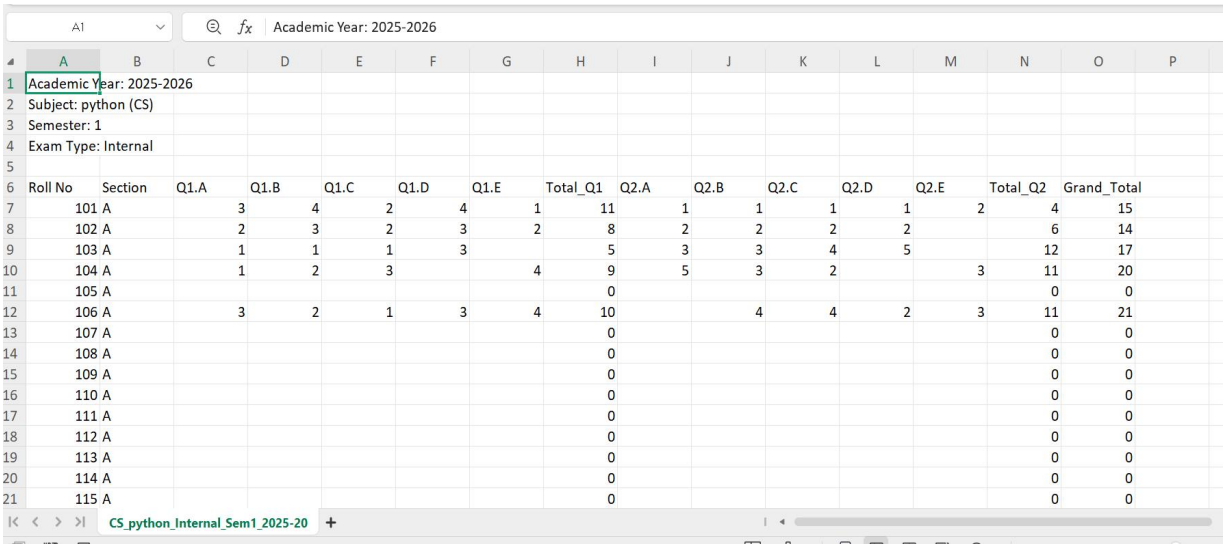


6. Question-wise Marks Entry Table



This screenshot illustrates the **question-wise marks entry interface**. Teachers enter sub-question marks per student, mark absentees, and view automatically calculated question totals and grand totals. The system enforces rules such as “any N out of K” during computation.

7. Exported Excel Report



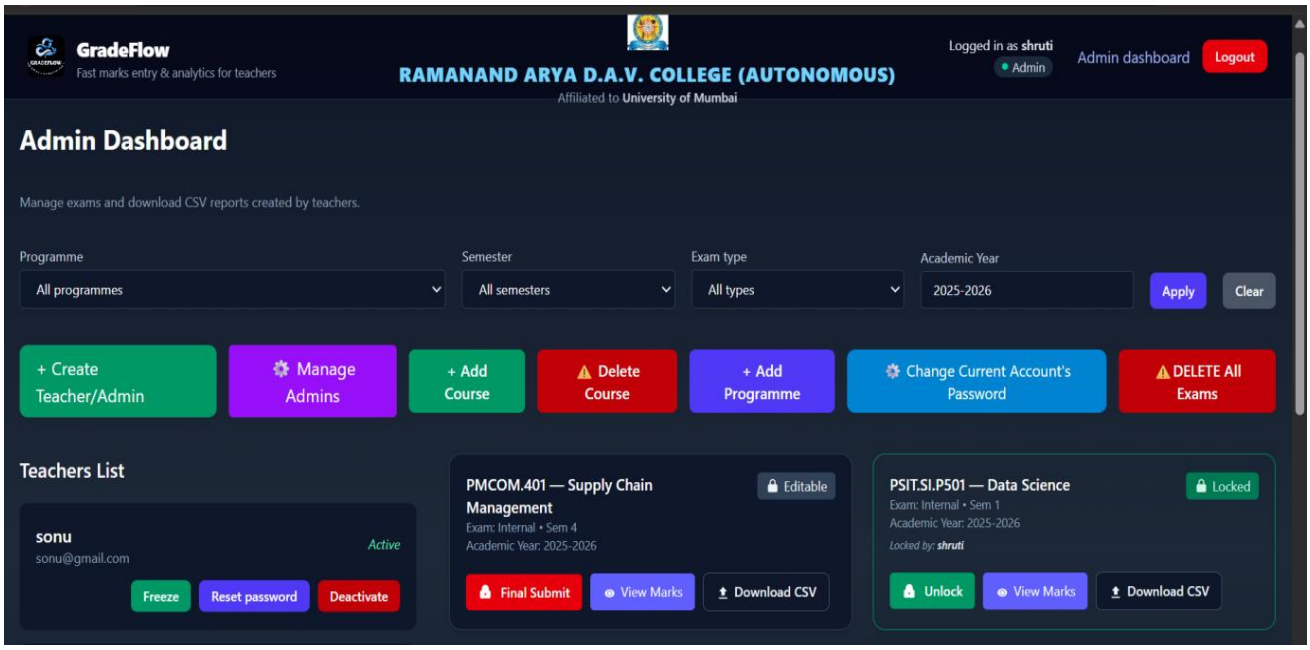
A1 | Academic Year: 2025-2026

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P
1	Academic Year: 2025-2026															
2	Subject: python (CS)															
3	Semester: 1															
4	Exam Type: Internal															
5																
6	Roll No	Section	Q1.A	Q1.B	Q1.C	Q1.D	Q1.E	Total_Q1	Q2.A	Q2.B	Q2.C	Q2.D	Q2.E	Total_Q2	Grand_Total	
7	101	A	3	4	2	4	1	11	1	1	1	1	2	4	15	
8	102	A	2	3	2	3	2	8	2	2	2	2		6	14	
9	103	A	1	1	1	3		5	3	3	4	5		12	17	
10	104	A	1	2	3		4	9	5	3	2		3	11	20	
11	105	A						0						0	0	
12	106	A	3	2	1	3	4	10		4	4	2	3	11	21	
13	107	A						0						0	0	
14	108	A						0						0	0	
15	109	A						0						0	0	
16	110	A						0						0	0	
17	111	A						0						0	0	
18	112	A						0						0	0	
19	113	A						0						0	0	
20	114	A						0						0	0	
21	115	A						0						0	0	

CS_python_Internal_Sem1_2025-20

This screenshot displays the **Excel report generated by GradeFlow**. It contains academic metadata, roll numbers, section details, question-wise marks, computed totals, and grand totals. The exported file serves as a clean, structured output for examination processing.

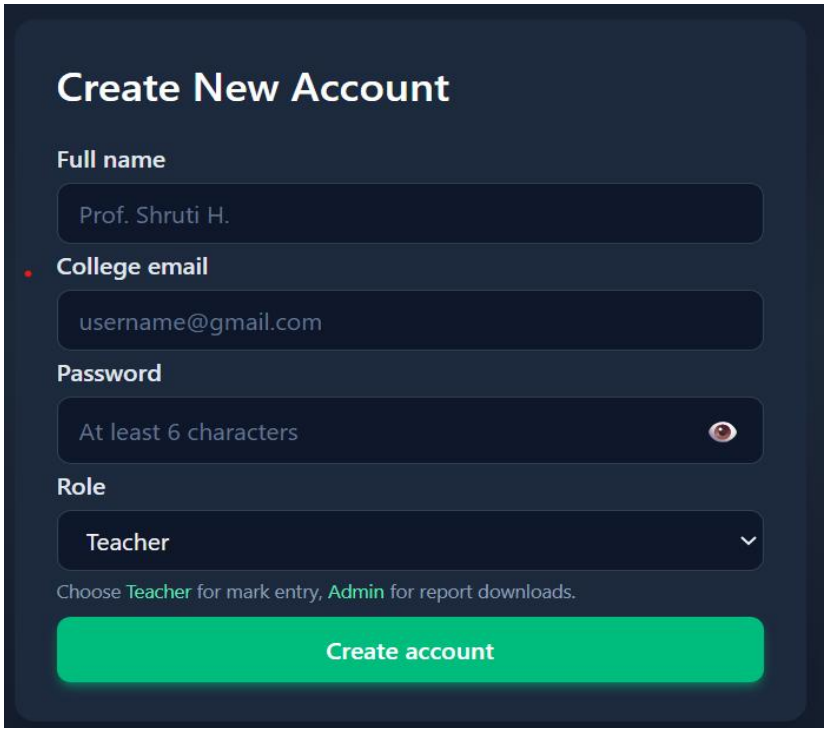
8. Admin Dashboard



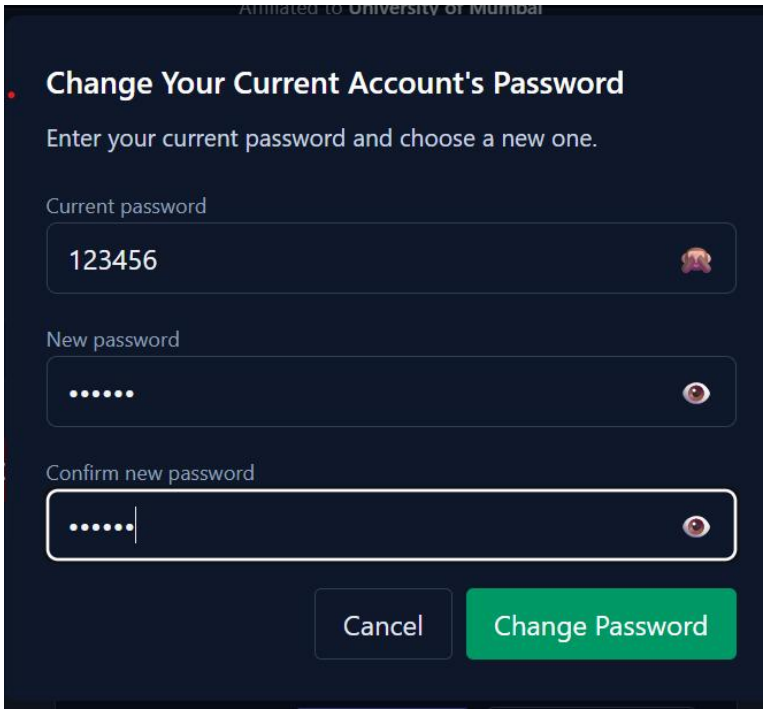
This screen is the Admin Dashboard in GradeFlow, used to manage programmes, courses, teachers, and exams. Administrators can filter exams by programme, semester, exam type, and academic year, create or delete courses and programmes, manage teacher accounts, and perform critical actions such as downloading reports or permanently deleting exams.

9. Create Teacher/Admin Account

This image displays the **account creation form** accessible to administrators. It allows creation of teacher or admin users by entering name, college email, password, and role. This ensures centralized user management and controlled system access.

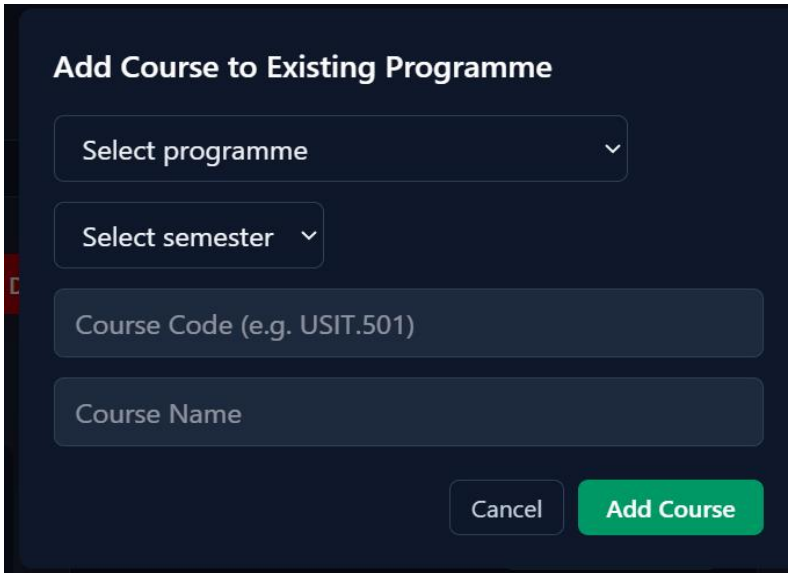


10. Change Password Interface



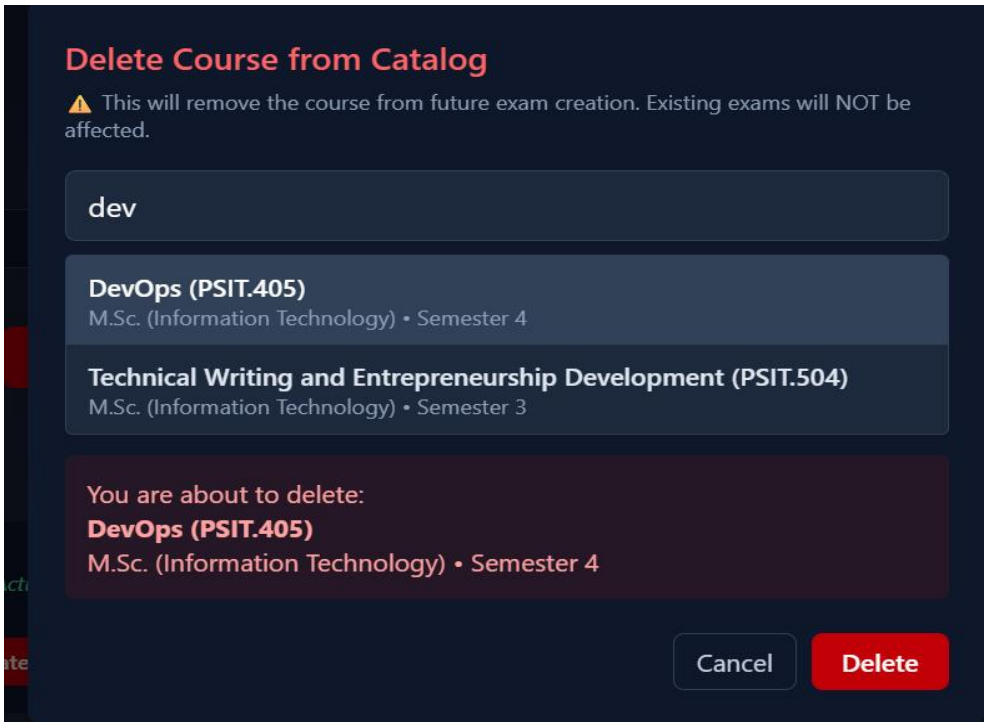
This screenshot shows the **password change dialog** for admin users. It requires the current password and confirmation of a new password. The feature supports secure credential management within the system.

11. Add Course to Existing Programme



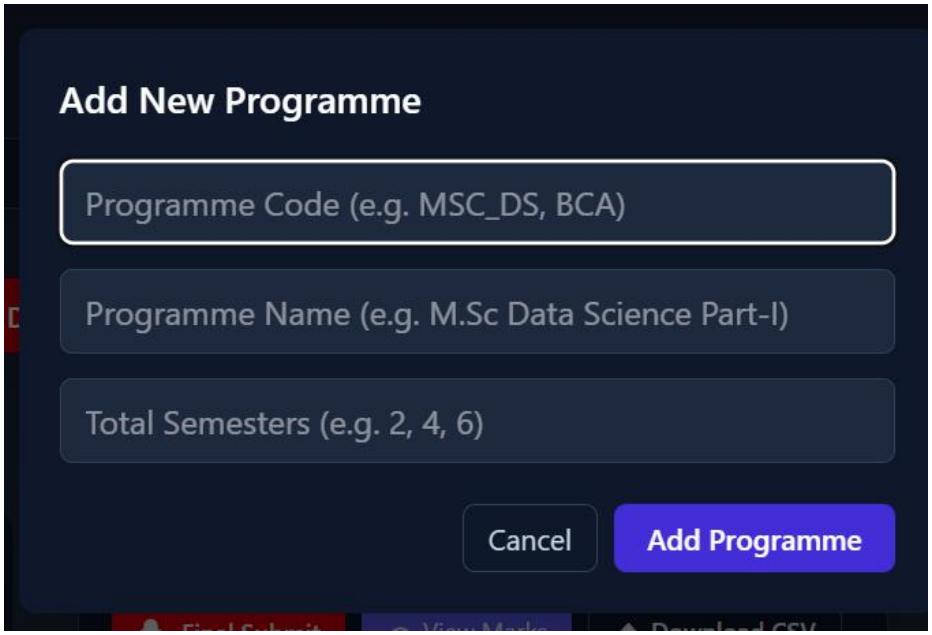
This screen allows the administrator to add a new course to an existing programme in GradeFlow. The user selects the programme and semester, then enters the course code and course name. On clicking **Add Course**, the course is saved and linked to the selected programme and semester, while **Cancel** exits without saving any changes.

12. Delete Course from catalog



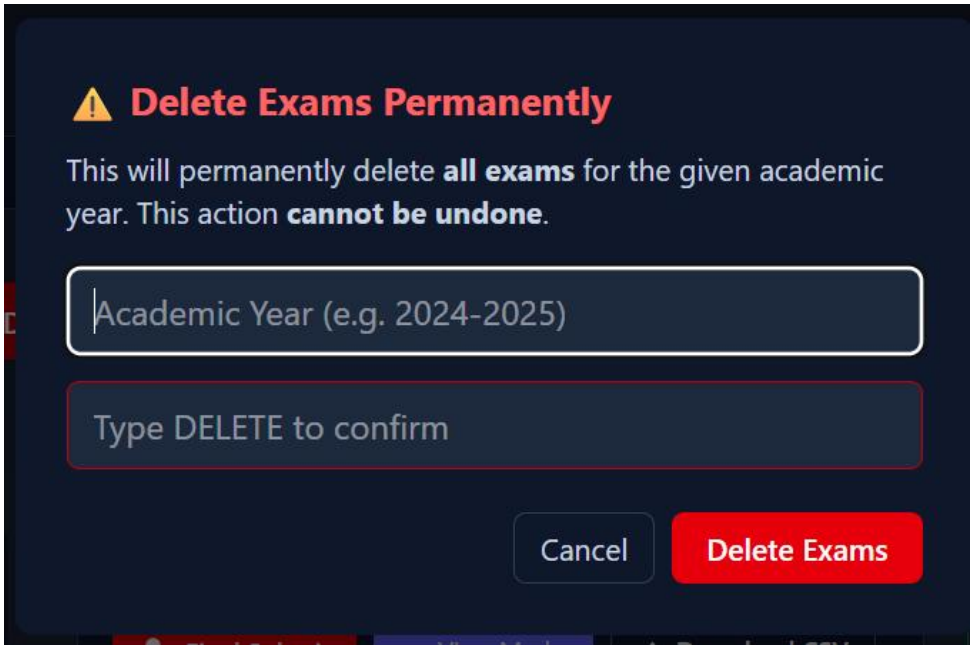
This screen allows the administrator to delete a course from the course catalog in GradeFlow. The user can search and select a course, after which a confirmation message clearly displays the selected course details. Deleting the course removes it from future exam creation while ensuring that any existing exams remain unaffected, and the action can be confirmed or canceled using the provided buttons.

13. Add new Programme



This screen is used to add a new academic programme in GradeFlow. The administrator enters the programme code, programme name, and the total number of semesters. Clicking **Add Programme** saves the programme in the system, while **Cancel** exits without making any changes.

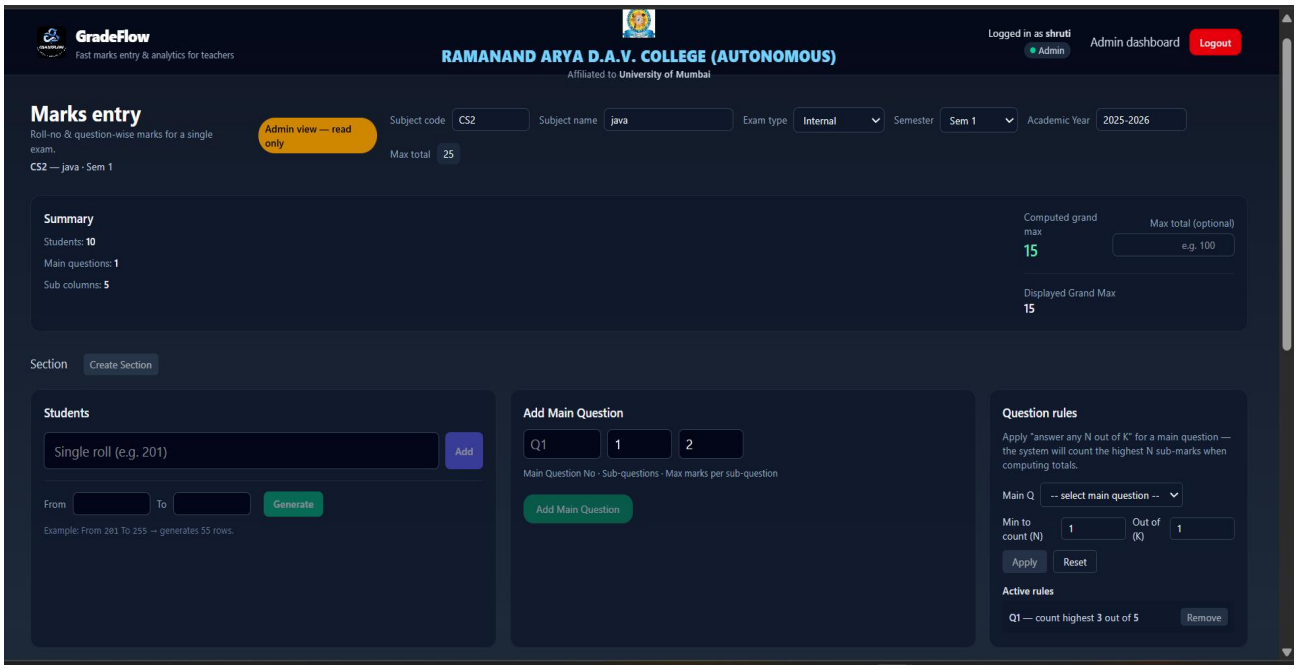
14. Delete All Exams



This screen allows the administrator to permanently delete all exams for a specified academic year in GradeFlow. The user must enter the academic year and type **DELETE** to confirm the action. Once confirmed, all exams for that year are removed permanently and the action cannot be undone, with options to proceed or cancel.

15. Admin View – Marks Entry (Read-Only)

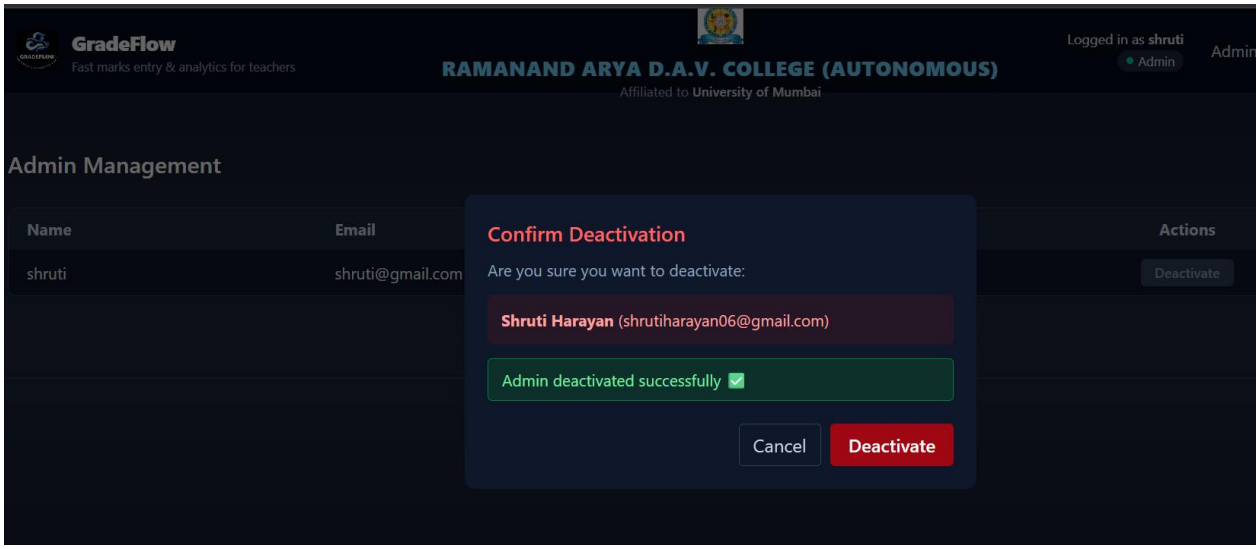
This image illustrates the **admin read-only view** of the marks entry screen. Administrators can review subject details, evaluation rules, and computed totals without modifying data. This ensures transparency while preserving data integrity.



16. Admin Review of Question-wise Marks

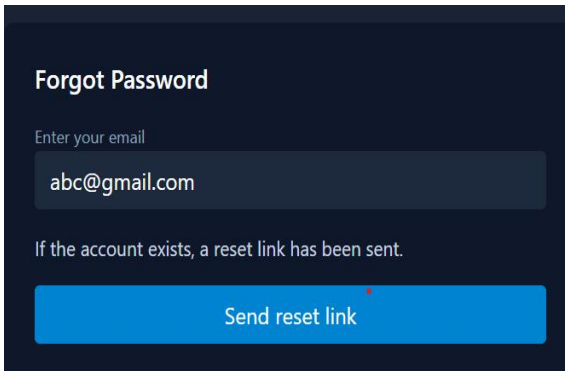
This screenshot displays the **question-wise marks table in admin mode**. It shows student roll numbers, sub-question marks, computed totals, and grand totals. Editing is disabled, reinforcing controlled review and audit functionality.

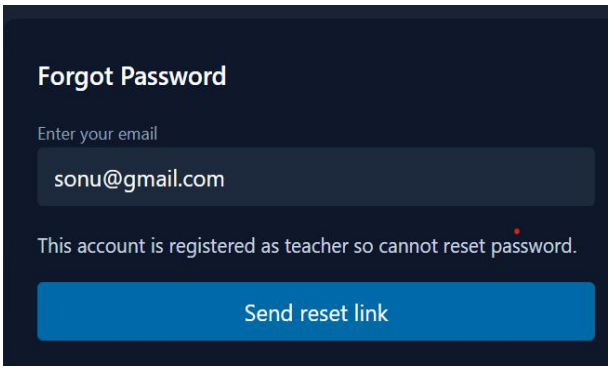
17. Admin Management

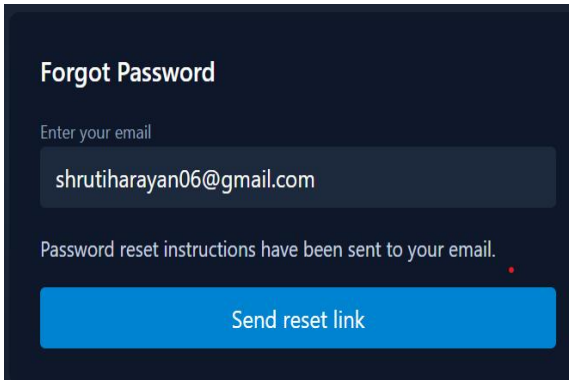


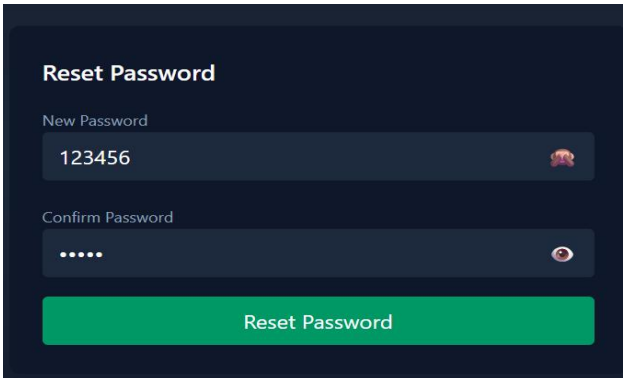
This screen represents the **Admin Management** module in GradeFlow, where existing admin accounts are listed with their names and email addresses. The administrator can deactivate other admin accounts using the provided action button, ensuring controlled access management. Upon selecting deactivate, a confirmation dialog appears to prevent accidental changes, and a success message is displayed once the admin account is successfully deactivated.

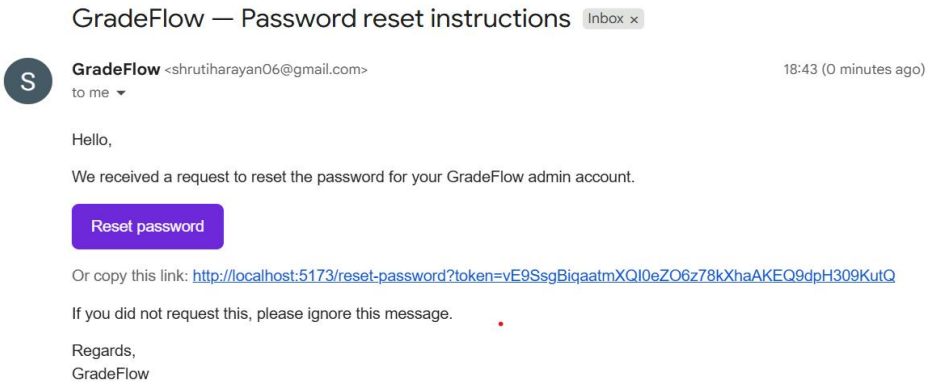
18. Forgot password







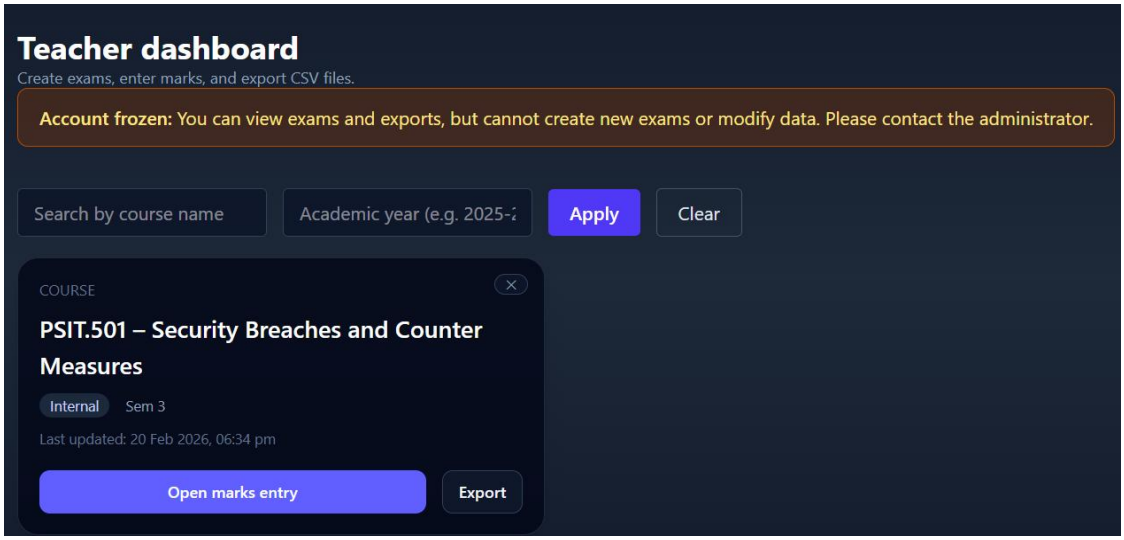




This screenshots represents the **Forgot Password and Reset Password** process in GradeFlow. Users enter their email to request a reset link, and if the account is valid and eligible (**only allowed for admins**), password reset instructions are sent via email. The email contains a secure link that opens the reset form, where the user enters and confirms a new password to complete the process. The system also shows appropriate messages for invalid requests or restricted account types to ensure secure password recovery.

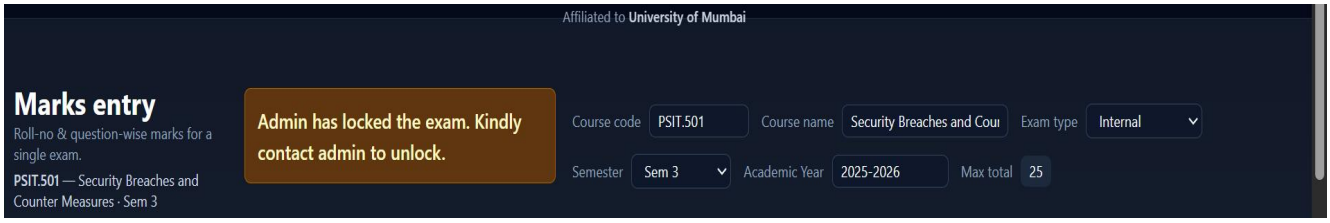
19. Teacher Account Freeze

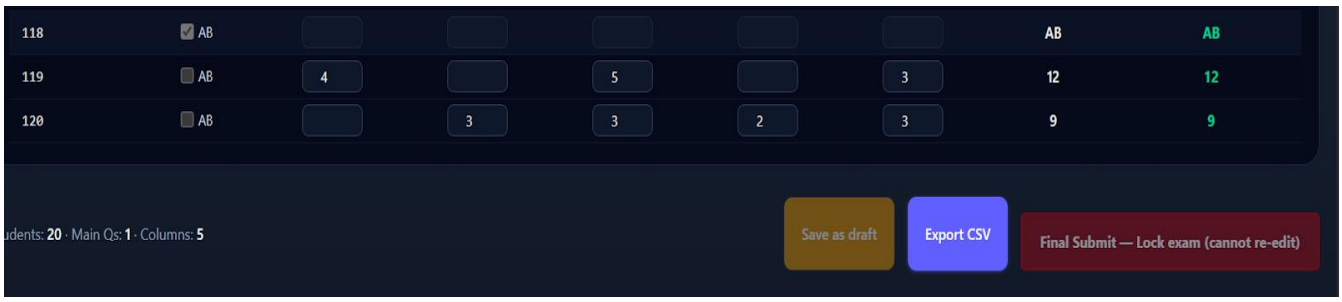
The administrator can temporarily freeze a teacher's account when necessary, such as after completion of evaluation or during an administrative review. Once frozen, the teacher is unable to create exams, modify existing exams, or enter and edit marks. However, the teacher can still log in to the system and download Excel reports. This feature ensures that the examination cell maintains full control over any further modifications to academic records.



20. Admin Locking an Exam

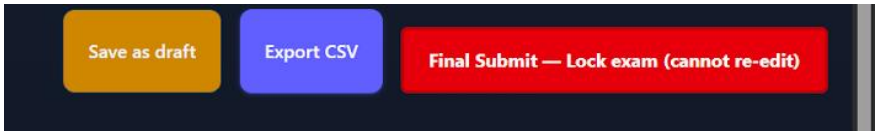
In GradeFlow, once marks are finalized, the administrator can lock the exam to prevent further modifications. After locking, teachers are unable to edit marks or add students, and the exam becomes read-only, with only the Excel download option remaining available. If any corrections are needed, the teacher must request the administrator to unlock the exam, ensuring that final control over mark changes remains with the examination cell.





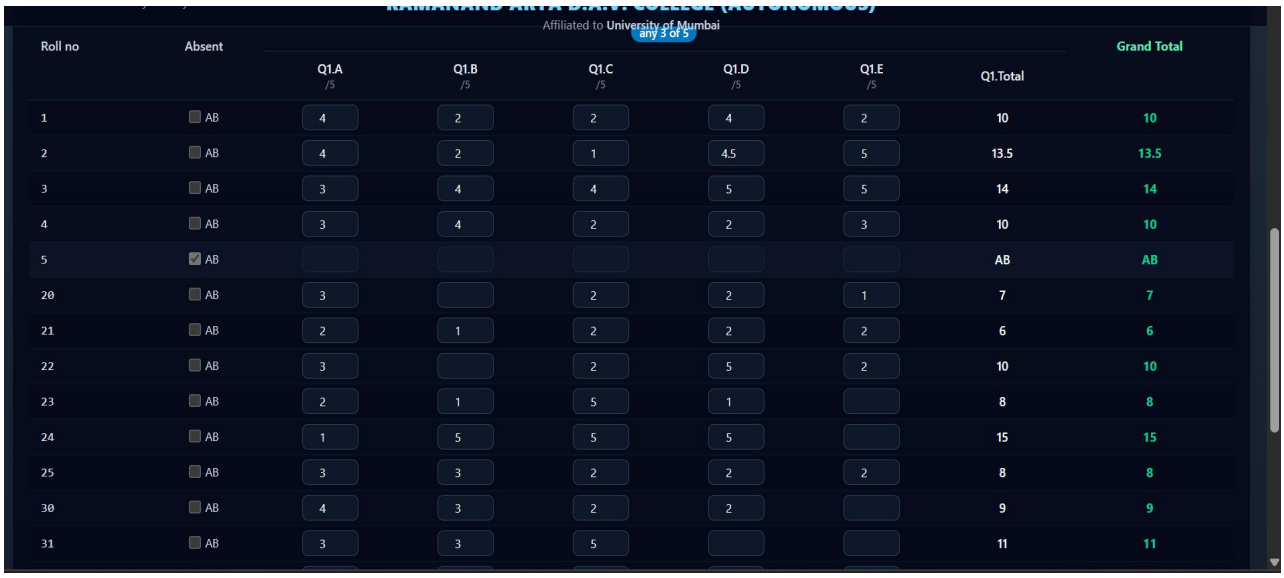
21. Draft vs Final Submission

In GradeFlow, teachers can initially save marks in Draft mode, allowing them to review and modify entries multiple times. Once the marks are verified, the teacher performs a Final Submit, after which the marks become locked and editing is disabled. Teachers can still view the data and download Excel reports, ensuring a single confirmed version of marks and preventing multiple conflicting final files.



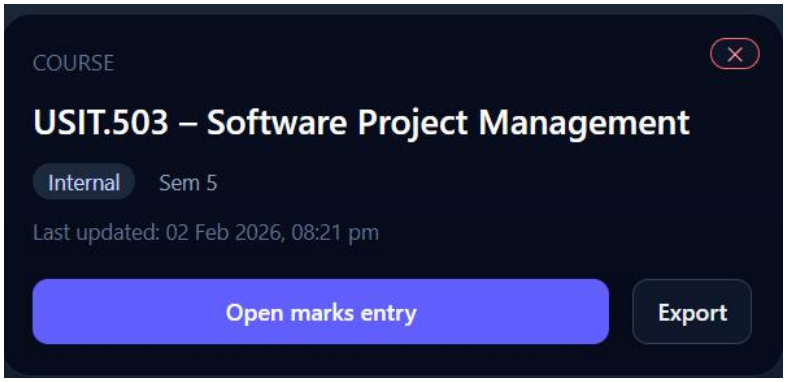
22. Combined marks view for admins

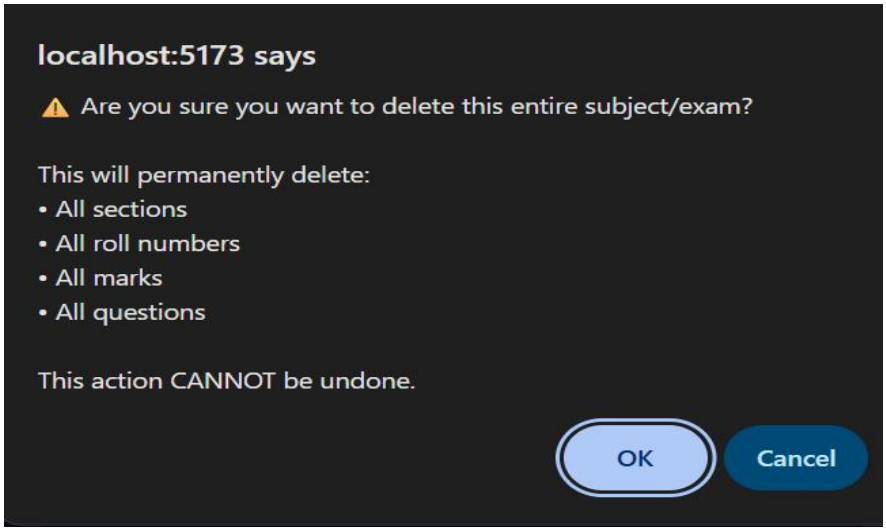
GradeFlow also provides a combined marks view for subjects handled by multiple teachers across different divisions. When the same subject, programme, semester, exam type, and academic year are shared but assigned to different teachers for separate divisions (e.g., Division A and Division B), the system automatically consolidates the marks. This allows the administrator to view and download a unified report for the entire class while still maintaining division-wise responsibility for marks entry.



Roll no	Absent	Q1.A /5	Q1.B /5	Q1.C /5	Q1.D /5	Q1.E /5	Q1.Total	Grand Total
1	AB	4	2	2	4	2	10	10
2	AB	4	2	1	4.5	5	13.5	13.5
3	AB	3	4	4	5	5	14	14
4	AB	3	4	2	2	3	10	10
5	AB						AB	AB
20	AB	3		2	2	1	7	7
21	AB	2	1	2	2	2	6	6
22	AB	3		2	5	2	10	10
23	AB	2	1	5	1		8	8
24	AB	1	5	5	5		15	15
25	AB	3	3	2	2	2	8	8
30	AB	4	3	2	2		9	9
31	AB	3	3	5			11	11

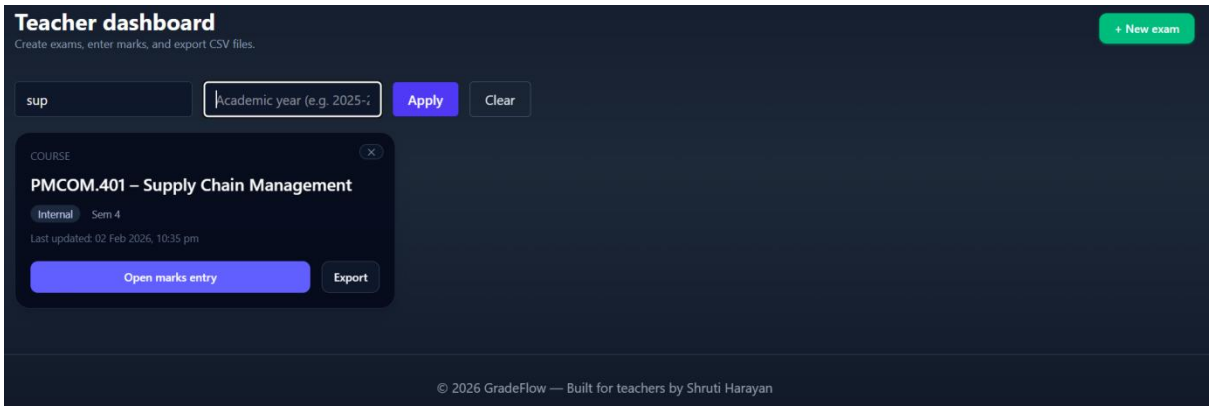
23. **Remove a subject from dashboard permanently**





This feature allows a teacher to permanently remove a subject/exam from the dashboard in GradeFlow. When the delete option is selected, the system shows a confirmation warning explaining that all related data, including sections, roll numbers, marks, and questions, will be permanently deleted and cannot be recovered. Once confirmed, the subject is removed completely from the teacher's dashboard and records.

24. Filter by course name and academic year



This feature allows teachers to filter exams on the dashboard by course name and academic year in GradeFlow. By entering a course keyword and specifying the academic year, teachers can quickly locate relevant exams without scrolling through the full list. The Apply button activates the filter, while Clear resets the search and displays all exams again.

25. EXAM FILTERING & SEARCH

Filters Section (Top Controls)

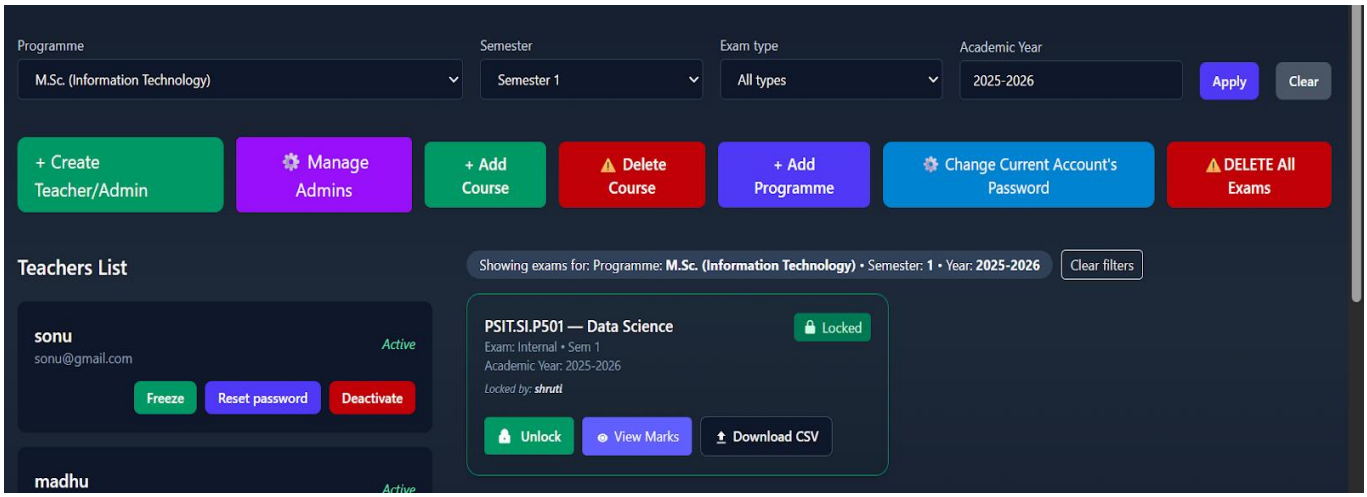
At the top of the dashboard, the admin can filter exams using:

- Programme
- Semester
- Exam type

- Academic year

This helps the examination cell quickly locate specific exams without searching manually.

After selecting filters, click **Apply** to display only relevant exams.



26. TEACHER MANAGEMENT

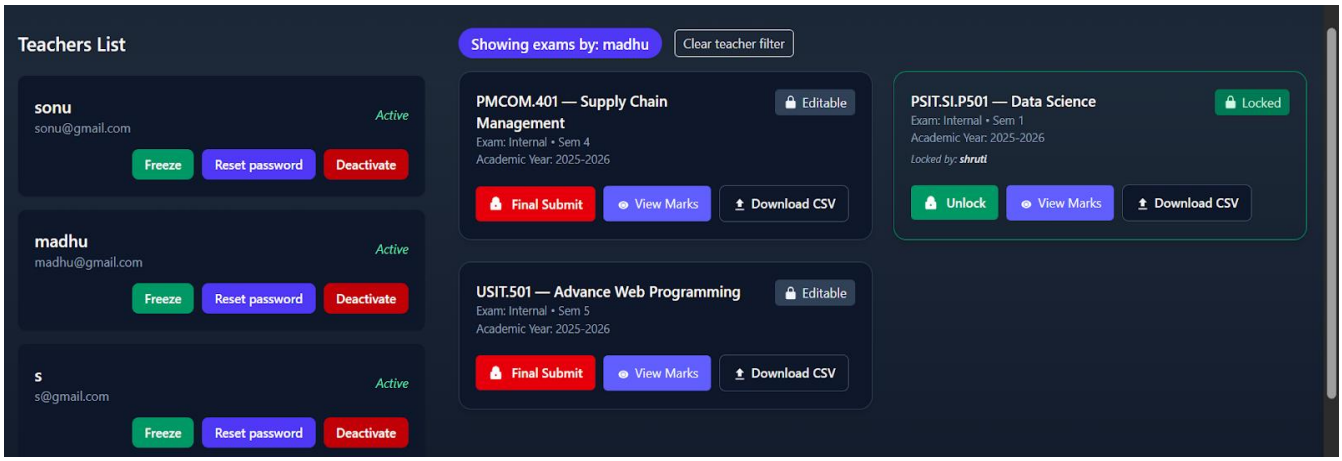
Teachers List Panel

This panel displays all registered teachers.

Admins can:

View teacher accounts
Filter exams created by a specific teacher
Control teacher access

Clicking a teacher shows only exams created by that teacher.



Teacher Control Buttons

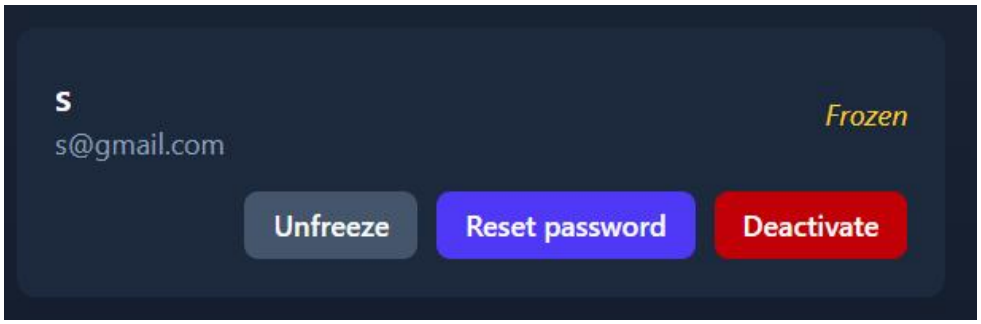
Freeze

Freezing temporarily stops a teacher from editing exams.

After freezing:

- Teachers cannot create or modify exams
- Teachers cannot enter marks
- Teachers can still log in
- Teachers can still download Excel reports

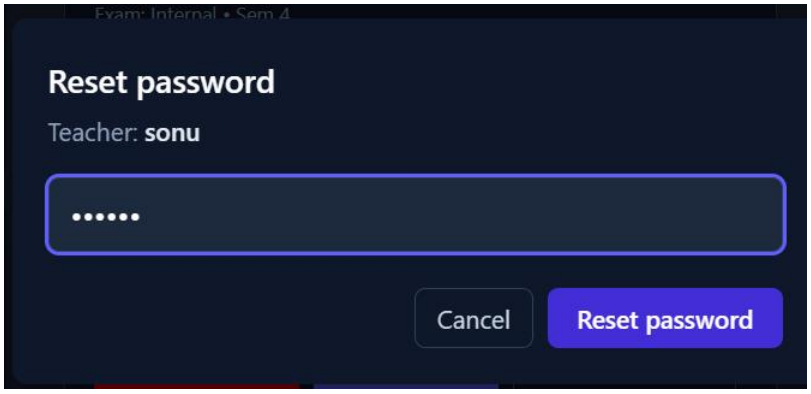
This is useful after evaluation completion or during administrative review.

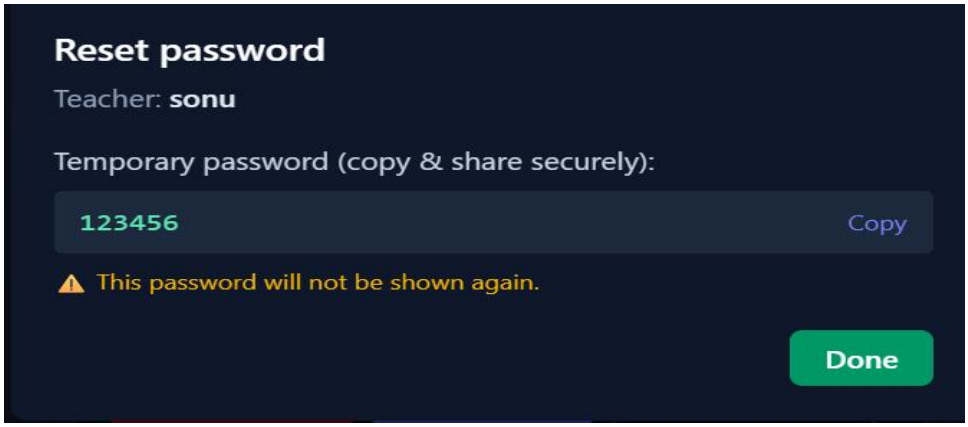


Reset Password

Reset Password allows the admin to generate new login credentials for teachers.

This ensures teachers regain access securely if credentials are lost.

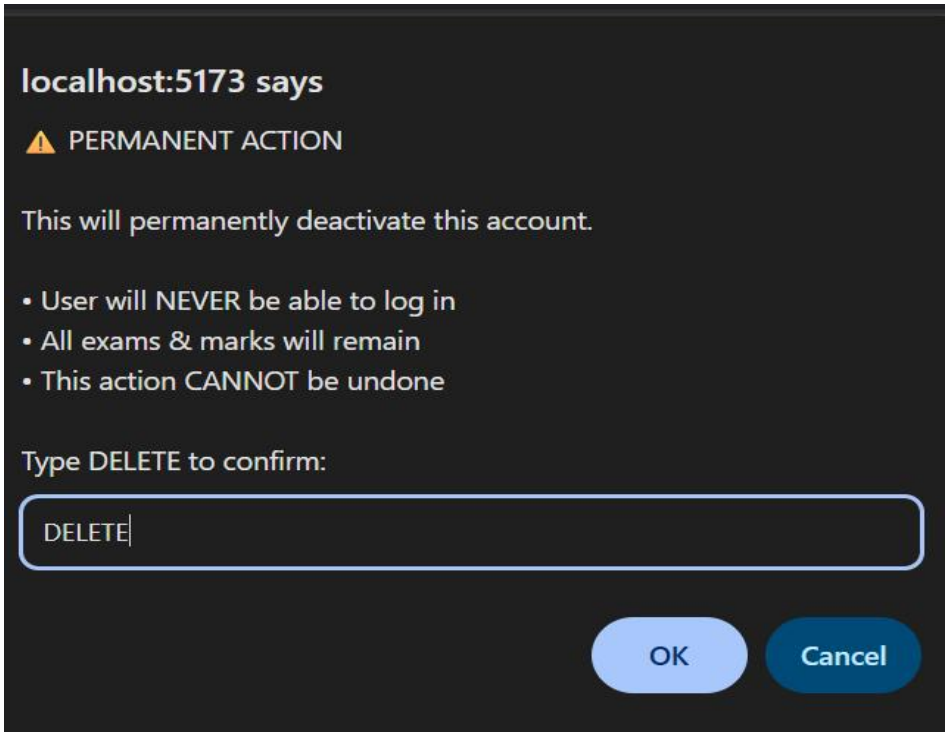




Deactivate

Deactivate removes system access without deleting the teacher's data or exams.

This is useful when staff leave or accounts must be disabled permanently.



27.Exam Summary and Marks Calculation Panel

The Summary panel displays an overview of the exam structure and total marks calculation.

It shows:

- **Students** – total number of students added for the exam
- **Main Questions** – number of main questions created
- **Sub Columns** – total number of sub-question fields for marks entry

On the right side:

- **Computed Grand Max** – the total marks automatically calculated based on all questions and sub-questions
- **Max Total (optional)** – allows the teacher to manually enter the official paper total if needed
- **Displayed Grand Max** – the final total that will appear in reports and exported Excel files

This panel helps teachers verify that the exam structure and marks total are correct before submitting marks.

Database Design and Tables

GradeFlow uses a relational database to store structured academic and workflow data. The following tables form the core of the system.

Users Table

The **users** table stores authentication and role-related information for all system users, including teachers and administrators. It supports login management, role-based access control, and account status tracking.

Field Description

- **id** – Unique identifier for each user in the system.
- **name** – Full name of the user.
- **email** – User's email address used for login and communication.
- **hashed_password** – Securely stored encrypted password for authentication.
- **role** – Defines whether the user is a teacher or administrator.
- **created_at** – Timestamp indicating when the account was created.
- **is_frozen** – Indicates whether the account is temporarily frozen by the admin, preventing grading actions.
- **is_deleted** – Soft-delete flag used to disable accounts without permanently removing records.

This table forms the foundation for user authentication, authorization, and workflow control within GradeFlow.

users		CREATE TABLE users (id INTEGER NOT NULL, name VARC
id	INTEGER	"id" INTEGER NOT NULL
name	VARCHAR(255)	"name" VARCHAR(255)
email	VARCHAR	"email" VARCHAR NOT NULL
hashed_password	VARCHAR	"hashed_password" VARCHAR
role	VARCHAR	"role" VARCHAR
created_at	DATETIME	"created_at" DATETIME DEFAULT CURRENT_TIMESTAMP
is_frozen	BOOLEAN	"is_frozen" BOOLEAN NOT NULL
is_deleted	BOOLEAN	"is_deleted" BOOLEAN NOT NULL DEFAULT 0

Subjects Catalog Table

The **subjects_catalog** table stores the master list of subjects offered under different programmes and semesters. It acts as the central reference for course creation, ensuring that exams are created only for valid and approved subjects.

Field Description

- **id** – Unique identifier for each subject entry in the catalog.
- **programme** – Name of the academic programme to which the subject belongs (e.g., B.Sc IT, M.Sc IT).
- **semester** – Indicates the semester in which the subject is taught.
- **subject_code** – Unique code assigned to the subject for identification and exam creation.
- **subject_name** – Full name of the subject.
- **is_active** – Flag indicating whether the subject is currently active and available for exam creation. Inactive subjects remain stored for historical reference but are not selectable.

This table ensures consistency in subject selection and prevents duplicate or incorrect subject entries during exam creation.

subjects_catalog		CREATE TABLE subjects_catalog (i
id	INTEGER	"id" INTEGER
programme	TEXT	"programme" TEXT NOT NULL
semester	INTEGER	"semester" INTEGER NOT NULL
subject_code	TEXT	"subject_code" TEXT NOT NULL
subject_name	TEXT	"subject_name" TEXT NOT NULL
is_active	INTEGER	"is_active" INTEGER DEFAULT 1

Questions Table

The **questions** table stores the structure of each exam by defining the individual questions or sub-questions associated with a specific exam. It enables fine-grained mark entry and supports flexible evaluation schemes at the question level.

Field Description

- **id** – Unique identifier for each question record.
- **exam_id** – References the exam to which the question belongs, linking questions to a specific subject assessment.
- **label** – Name or identifier of the question (e.g., Q1, Q2(a), Section A).
- **max_marks** – Maximum marks allocated to that question or sub-question.
- **order** – Indicates the sequence of the question in the exam, ensuring proper display and structured mark entry.

This table allows GradeFlow to represent detailed exam structures and supports accurate computation of totals based on individual question marks.

questions		CREATE TABLE questions (id INTEG
id	INTEGER	"id" INTEGER NOT NULL
exam_id	INTEGER	"exam_id" INTEGER
label	VARCHAR	"label" VARCHAR NOT NULL
max_marks	INTEGER	"max_marks" INTEGER NOT NULL
order	INTEGER	"order" INTEGER

Students Table

The **students** table stores the list of students associated with a particular exam. It enables the system to maintain exam-wise student records and track their attendance status during evaluation.

Field Description

- **id** – Unique identifier for each student record in the system.
- **exam_id** – References the exam for which the student is registered, linking the student to a specific assessment.
- **roll_no** – Roll number of the student, used for identification during marks entry and reporting.
- **absent** – Boolean flag indicating whether the student was absent for the exam. If marked absent, the system can restrict mark entry and reflect the status in reports.

This table ensures that GradeFlow maintains structured student lists for each exam and supports accurate mark recording and attendance tracking.

students		CREATE TABLE "students" (id INTE
id	INTEGER	"id" INTEGER
exam_id	INTEGER	"exam_id" INTEGER NOT NULL
roll_no	INTEGER	"roll_no" INTEGER NOT NULL
absent	BOOLEAN	"absent" BOOLEAN DEFAULT 0

Programmes Table

The **programmes** table stores information about the academic programmes offered by the institution. It defines the structure of each programme, including its name, code, and total number of semesters, and serves as a reference for organizing subjects and exams within GradeFlow.

Field Description

- **id** – Unique identifier for each programme in the system.
- **name** – Official name of the academic programme (e.g., B.Sc IT, M.Com). This field is unique to avoid duplicate programme entries.
- **total_semesters** – Total number of semesters in the programme, used to structure subjects and exams appropriately.
- **semester_start** – Indicates the starting semester numbering for the programme, useful for programmes that begin from a specific term.
- **programme_code** – Short code representing the programme, used for identification and referencing in the system.

This table provides the foundational academic structure that connects subjects, exams, and student evaluations within GradeFlow.

programmes		CREATE TABLE programmes (id INTEGE
id	INTEGER	"id" INTEGER
name	TEXT	"name" TEXT NOT NULL UNIQUE
total_semesters	INTEGER	"total_semesters" INTEGER NOT NULL
semester_start	INTEGER	"semester_start" INTEGER DEFAULT 1
programme_code	TEXT	"programme_code" TEXT

Password Resets Table

The **password_resets** table stores temporary reset tokens used during the password recovery process. It enables secure password reset functionality by linking a user account with a time-bound reset token.

Field Description

- **id** – Unique identifier for each password reset request.
- **user_id** – References the user requesting the password reset, linking the token to a specific account.
- **token** – Unique reset token generated by the system and sent to the user via email for verification.
- **expires_at** – Timestamp indicating when the reset token expires. After this time, the token becomes invalid to maintain security.

This table ensures secure handling of password recovery requests and prevents unauthorized password changes by enforcing token validation and expiry checks.

password_resets		CREATE TABLE password_resets(i
id	INTEGER	"id" INTEGER
user_id	INTEGER	"user_id" INTEGER NOT NULL
token	TEXT	"token" TEXT NOT NULL UNIQUE
expires_at	TEXT	"expires_at" TEXT NOT NULL

Marks Table

The **marks** table stores the detailed marks obtained by students for each question or sub-question in an exam. It forms the core evaluation dataset of GradeFlow and enables fine-grained computation of totals and aggregates.

Field Description

- **id** – Unique identifier for each marks record.
- **exam_id** – References the exam for which the marks are recorded, linking the entry to a specific assessment.
- **student_id** – References the student to whom the marks belong.
- **question_id** – References the question or sub-question being evaluated, allowing marks to be stored at a detailed level.
- **marks** – Actual marks obtained by the student for that question.
- **section_id** – Identifies the section or division of the exam, enabling the system to manage marks across multiple sections or evaluators.

This table is central to GradeFlow's rule-based computation process, as it stores the granular data used to calculate totals, apply aggregation rules, and generate final reports.

marks		CREATE TABLE marks (i
id	INTEGER	"id" INTEGER NOT NULL
exam_id	INTEGER	"exam_id" INTEGER
student_id	INTEGER	"student_id" INTEGER
question_id	INTEGER	"question_id" INTEGER
marks	INTEGER	"marks" INTEGER
section_id	INTEGER	"section_id" INTEGER

Exams Table

The **exams** table stores the core metadata of each assessment created in GradeFlow. It defines the subject, programme context, evaluation type, academic year, and workflow state of the exam, acting as the central entity that links students, questions, and marks.

Field Description

- **id** – Unique identifier for each exam record.
- **subject_code** – Code of the subject for which the exam is created.
- **subject_name** – Name of the subject associated with the exam.
- **exam_type** – Indicates the type of assessment (e.g., Internal, External).
- **semester** – Semester in which the exam is conducted.
- **students_count** – Stores the total number of students registered for the exam.
- **created_by** – References the user who created the exam, typically a teacher.
- **is_locked** – Boolean flag indicating whether the exam has been locked by the administrator, preventing further modifications.
- **created_at** – Timestamp recording when the exam was created.
- **updated_at** – Timestamp storing the last modification time of the exam.
- **academic_year** – Specifies the academic year for which the exam is conducted.
- **locked_by** – References the administrator who locked the exam.
- **question_rules** – Stores rule definitions used for aggregation of marks (e.g., best N out of M).
- **programme** – Name of the academic programme under which the exam is conducted.

This table serves as the central coordination point of GradeFlow, linking together students, questions, marks, and workflow controls, and ensuring that each exam follows a structured and trackable lifecycle.

exams		CREATE TABLE exams (id INTEGER NOT NULL, subject_c
id	INTEGER	"id" INTEGER NOT NULL
subject_code	VARCHAR	"subject_code" VARCHAR NOT NULL
subject_name	VARCHAR	"subject_name" VARCHAR NOT NULL
exam_type	VARCHAR	"exam_type" VARCHAR NOT NULL
semester	INTEGER	"semester" INTEGER NOT NULL
students_count	INTEGER	"students_count" INTEGER
created_by	INTEGER	"created_by" INTEGER
is_locked	BOOLEAN	"is_locked" BOOLEAN NOT NULL
created_at	DATETIME	"created_at" DATETIME DEFAULT CURRENT_TIMESTAMP
updated_at	DATETIME	"updated_at" DATETIME
academic_year	VARCHAR(255)	"academic_year" VARCHAR(255) DEFAULT '2025-2026'
locked_by	number	"locked_by" number
question_rules	TEXT	"question_rules" TEXT
programme	TEXT	"programme" TEXT NOT NULL DEFAULT ''

Exam Sections Table

The **exam_sections** table manages division-wise allocation of an exam to different teachers. It allows GradeFlow to support scenarios where the same subject exam is handled by multiple teachers for different student sections or roll number ranges.

Field Description

- **id** – Unique identifier for each exam section entry.
- **exam_id** – References the exam to which this section belongs, linking the division to a specific assessment.
- **teacher_id** – References the teacher responsible for evaluating this section.
- **section_name** – Name of the division or section (e.g., A, B, C).
- **roll_start** – Starting roll number assigned to the section.
- **roll_end** – Ending roll number assigned to the section.
- **is_locked** – Indicates whether this particular section has been locked, preventing further edits to marks within that division.
- **locked_by** – References the administrator who locked the section.
- **created_at** – Timestamp recording when the section assignment was created.

This table enables GradeFlow to distribute evaluation responsibilities among multiple teachers while still allowing the administrator to obtain a unified view of marks across all sections of the same exam.

exam_sections		CREATE TABLE exam_sections (id IN
id	INTEGER	"id" INTEGER
exam_id	INTEGER	"exam_id" INTEGER NOT NULL
teacher_id	INTEGER	"teacher_id" INTEGER NOT NULL
section_name	TEXT	"section_name" TEXT
roll_start	INTEGER	"roll_start" INTEGER NOT NULL
roll_end	INTEGER	"roll_end" INTEGER NOT NULL
is_locked	BOOLEAN	"is_locked" BOOLEAN DEFAULT 0
locked_by	INTEGER	"locked_by" INTEGER
created_at	TEXT	"created_at" TEXT

System Architecture

1. Overview

The GradeFlow system is designed as a **web-based, client–server application** that supports structured mark entry, automated grade computation, and controlled submission workflows. The architecture follows a **modular and layered design**, ensuring separation of concerns between the user interface, application logic, and data storage. This approach improves maintainability, scalability, and reliability of the system.

At a high level, GradeFlow consists of three primary layers:

- Presentation Layer (Frontend)
- Application Layer (Backend)
- Data Layer (Database)

The architecture follows a **three-tier client–server model** combined with **REST-based service communication**. This architectural style was selected to ensure modularity, centralized rule enforcement, and compatibility with web deployment environments.

2. Architectural Design

2.1 Presentation Layer (Frontend)

The Presentation Layer provides the user interface through which teachers and administrators interact with the system. It is implemented as a **web-based client application** accessible via standard browsers.

Key responsibilities:

- User authentication and role-based UI rendering
- Mark entry at sub-question level
- Display of computed totals and grand totals
- Workflow actions such as *Save as Draft* and *Final Submit*
- Visual feedback for locked or disabled states

The frontend ensures that users can easily transition from spreadsheet-based workflows while minimizing the possibility of incorrect data entry.

2.2 Application Layer (Backend)

The Application Layer contains the core business logic of GradeFlow. It is responsible for processing user requests, enforcing grading rules, and managing workflow states.

Key responsibilities:

- Handling API requests from the frontend
- Validating mark entries and grading rules
- Applying rule-based aggregation logic (e.g., best N out of M)
- Managing submission states (draft, finalized, locked)
- Enforcing role-based permissions

This layer ensures that all computations and state transitions are handled centrally, eliminating reliance on client-side calculations.

2.3 Data Layer (Database)

The Data Layer stores all persistent data required by the system. A **relational database model** is used to maintain structured and consistent storage of academic data.

Key responsibilities:

- Storing student, subject, and assessment metadata
- Persisting sub-question marks and computed totals
- Recording submission status for each assessment
- Supporting retrieval of authoritative mark data for examination processing

By centralizing data storage, GradeFlow maintains a single source of truth for all grading information.

2.4 Technology Mapping to Architecture

Each architectural layer is mapped to specific technologies used in implementation. The Presentation Layer is built using React with TypeScript and Tailwind CSS, enabling responsive and structured user interaction. The Application Layer is implemented using FastAPI in Python, which manages API routing, validation, and business logic execution. The Data Layer uses SQLite as a relational database for structured storage of grading data. This clear mapping ensures that architectural design decisions directly translate into implementation components.

3. Component Interaction

The interaction between system components follows a request–response model:

- The user performs an action (e.g., entering marks or submitting final results) through the frontend.
- The frontend sends a request to the backend via RESTful APIs.
- The backend validates the request, applies grading rules, and updates the database.
- The backend returns a response indicating success or failure.
- The frontend updates the UI based on the response and current workflow state.

This controlled interaction ensures data consistency and prevents unauthorized or unintended modifications.

4. Workflow Enforcement Mechanism

A key architectural feature of GradeFlow is its **workflow enforcement mechanism**. Unlike spreadsheet-based systems, GradeFlow explicitly models the lifecycle of grading data.

- **Draft State:** Marks can be entered and modified freely.
- **Final State:** Marks are validated and finalized by the teacher.

- **Locked State:** Further modifications are disallowed.

Workflow state checks are enforced at the application layer to ensure that invalid actions are rejected regardless of the client interface.

5. Security and Access Control

GradeFlow implements **role-based access control** at both the frontend and backend layers.

- Teachers can enter and submit marks within permitted states.
- Administrators can manage system-level permissions and freeze evaluations.

Security controls are implemented server-side to prevent unauthorized access or bypassing of workflow restrictions.

System Reliability Considerations

To ensure reliability, GradeFlow performs validation checks before committing data changes and prevents partial or inconsistent updates through controlled transaction handling. Backend validation ensures that incorrect or incomplete mark data cannot be stored, while workflow checks prevent unauthorized transitions between draft, final, and locked states. These mechanisms reduce the risk of data corruption and ensure the integrity of assessment records.

6. Scalability and Maintainability

The modular architecture allows GradeFlow to scale horizontally by separating concerns:

- The frontend can be deployed independently of the backend.
- The backend can be extended to support additional grading rules or workflows.
- The database schema supports normalization and future extensions.

This design ensures that the system can be adapted to varying institutional requirements without major architectural changes. Although the current implementation uses SQLite for simplicity, the architecture is database-agnostic and can be migrated to enterprise databases such as PostgreSQL or MySQL without structural redesign. This ensures readiness for institutional-scale deployment.

7. Architectural Summary

The system architecture of GradeFlow provides a structured and reliable foundation for academic grading workflows. By combining a layered architecture with centralized rule enforcement and workflow control, the system addresses the limitations of spreadsheet-based grading and generic LMS gradebooks. The architecture ensures accuracy, consistency, and clarity in the management of academic assessment data.

Conclusion

This study addressed the challenges associated with spreadsheet-based grading practices commonly used in higher education institutions. Manual mark entry, reliance on user-defined formulas, repeated re-submission of revised Excel files, and the absence of structured workflow control were identified as major sources of error and administrative confusion in examination processes.

To overcome these limitations, the GradeFlow system was designed and developed as a dedicated grading platform focused on automated computation and controlled submission workflows. By supporting fine-grained mark entry and rule-based aggregation, the system eliminates manual calculation errors and ensures consistent application of evaluation rules across all assessments. The introduction of explicit draft and final states provides clarity in the grading lifecycle and prevents ambiguity arising from multiple revised submissions.

The layered system architecture, combined with centralized business logic and role-based access control, ensures reliability, maintainability, and scalability of the proposed solution. Scenario-based testing demonstrated that GradeFlow effectively reduces computational inaccuracies and administrative overhead while improving workflow transparency for both teachers and examination committees.

In conclusion, GradeFlow provides a practical and efficient alternative to spreadsheet-based grading systems. By aligning digital tool capabilities with real-world academic evaluation workflows, the system contributes to improved accuracy, consistency, and clarity in academic assessment management.

Future Scope and Enhancements

While GradeFlow effectively addresses the challenges associated with spreadsheet-based grading and workflow ambiguity, there are several opportunities for future enhancement that can further improve its functionality and applicability in academic environments.

One potential enhancement is the **integration of advanced analytics and reporting features**. Future versions of the system could provide analytical dashboards for examination committees, offering insights such as mark distributions, pass–fail statistics, and subject-wise performance trends. Such features would support data-driven decision-making while maintaining the integrity of finalized marks.

Another area for enhancement is **expanded support for institutional policies and evaluation schemes**. Although GradeFlow currently supports rule-based aggregation, additional configuration options such as weighted components, moderation rules, and conditional adjustments could be introduced to accommodate a wider range of academic regulations across institutions.

The system can also be extended to support **interoperability with existing academic platforms**, such as learning management systems or student information systems. Controlled data exchange mechanisms would allow institutions to leverage GradeFlow's grading capabilities while maintaining consistency with existing digital infrastructure.

Future development may also focus on **enhancing usability and accessibility**. Features such as customizable templates for different subjects, bulk mark entry improvements, and accessibility support for diverse users can further simplify the grading process for educators.

Finally, **scalability and deployment enhancements** can be explored, including support for cloud-based deployment and multi-department usage. Such improvements would enable GradeFlow to serve larger institutions and adapt to varying academic structures without significant architectural changes.

References

Anne Mesny , Isabelle Roberge-Maltais , Anais Galy (2026), "*Innovative assessment and grading practices in higher education: a critical exploration for management educators*", The International Journal of Management Education 24 (2026) 101307.

Al-Ali, Maytha (2024), "*Assessment Management in Higher Education*". All Works. 6593.

Abdul Muis Mappalotteng , *Fathahillah* Fathahillah , and *Muhammad* Anas Punggawa, "*WEB-BASED STUDENT ACADEMIC GRADE PROCESSING INFORMATION SYSTEM*", ITM Web of Conferences 58, 03006 (2024)

Ms. Nishu Sethi , and Dr. Anshu Malhotra (2023), "*Efficiency Engine: Designing and Implementing an Academic Management System "*, International Journal of Innovative Research in Engineering and Management (IJIREM) ISSN (Online): 2350-0557, Volume-10, Issue-3, June 2023

Emma Mayhew, Vicki Holmes, Madeleine Davies and Yota Dimitriadi(2022), "*Online submission, feedback and grading of assessment: what do academic staff really think?*" *Research in Learning Technology* Vol. 30, 2022

Sita Ramnan, Dr. Kavita Kalyandurgmath (2019), "*Variation in the Grading System: Boon or Bane Standardizing the Assessment Reports: Overcoming Variations in Grading System*", Journal of Advances and Scholarly Researches in Allied Education, E-ISSN: 2230-7540 Volume 16, Issue No. 2, Feb 2019.

Powell, S. G., Baker, K. R., & Lawson, B. (2009). "*Errors in Operational Spreadsheets."* Decision Support Systems.

Panko, R. R. (2005). "*What We Know About Spreadsheet Errors*." Journal of End User Computing.

Rajalingham, K., Chadwick, D., Knight, B. (2005). "*Classification of Spreadsheet Errors"*. European Spreadsheet Risks Interest Group.

Jonathan P. Caulkins, Erica Layne Morrison, and Timothy Weidemann, (2005), "*Spreadsheet Errors and Decision Making: Evidence from Field Interviews.*"